大语言模型推理优化技术实训报告

015 组

张** 张* 熊**

摘要

随着大语言模型在智能客服、代码生成和边缘计算等场景的广泛应用，其推理效率成为制约部署可行性的关键瓶颈。为应对高显存占用、低吞吐量与长延迟等问题，本次实训我们小组从零搭建了 ZLLM 轻量化推理框架，实现了 KV Cache 与分布式推理架构优化，并以 VLLM 为基础搭建了高效的大语言模型推理系统，此外，我们还深入研究了动态批处理与 PagedAttention 显存分页机制、以及模型量化与压缩技术。实现了全部基础功能以及加分项目，并深入研究分析了推理框架的性能瓶颈和探索可能的优化方向，截止提交时间我们在实时榜单上取得了**综合分第二，TTFT，P95 等指标相对更加优异**的好成绩。

关键词：KV Cache、动态批处理、PagedAttention、模型量化、异步任务调度、分布式推理

目录

大语言模型推理优化技术实训报告 1

摘要 1

第一章 引言：实训技术路线概述 5

1.1 行业背景与发展现状 错误！未定义书签。

1.2 推理性能瓶颈分析 错误！未定义书签。

1.3 全链路优化路径 错误！未定义书签。

1.4 报告结构说明 错误！未定义书签。

2.1 ZLLM 框架搭建 9

图 1 ZLLM 整体框架 10

2.1.2 核心优化层 10

2.1.3 推理引擎层 10

2.1.4 应用层 / 测评层 11

2.2 KV Cache 工作原理与设计 11

2.2.1 原理 11

2.2.2 设计 12

2.2.3 优化效果 15

2.3 分布式推理实现与优化 16

2.3.1 实现框架 16

2.3.2 推理优化尝试 17

2.4 vLLM 系统优化 19

2.4.2 异步任务调度 21

2.4.3 优化效果 24

3.1 动态 Batch 技术不同策略的探索 24

3.1.1 FIFODynamicBatchScheduler 25

3.1.3 Aging BatchScheduler 29

3.1.4 ContinuousBatchScheduler 30

3.1.6 方法对比 34

3.1.7 实验结果及分析 34

3.2 从 0 手搓 PagedAttention ——核心原理与数据结构设计 35

3.2.1 核心架构 35

3.2.2 物理池的设计 36

3.2.3 Block Table(映射表)的设计 36

3.2.4 写入操作（write_kv） 37

3.2.5 读取和计算：gather_kv 37

3.2.6 FlashAttention 优化 37

3.2.7 实验结果 38

3.3 CPU/GPU 异构推理的自主探索 39

3.3.1 层切分异构 40

➢ 引入 CPU 端高性能算子库 (计算加速) 42

3.4 总结与心得体会 43

第四章 模型量化与压缩技术 45

4.1 引言 45

4.2 量化技术原理与分类体系 46

（1）量化基本原理：从浮点到整数的映射机制 46

(2) 按阶段划分：PTQ 与 QAT 的技术特性对比 47

(3) 按粒度划分：逐张量、逐通道与逐组量化的精度-效率权衡 48

(4) 量化过程中的误差来源与缓解策略 50

4.3 主流量化算法与工具链生态分析 51

（1）GPTQ：逐层误差最小化的权重量化 51

（2）AWQ：激活感知的权重保护策略 52

（3）bitsandbytes 与 QLoRA：低资源微调的新范式 53

（4）GGUF：端侧部署的轻量化解决方案 54

4.4 实训结果与多维性能评估 55

（1）不同量化配置下的宏观趋势分析 56

4.5 改进量化策略与实验验证 59

（1）敏感层识别：为何下投影层不宜过早量化 59

（2）分步量化设计：三阶段渐进式量化框架 61

（3）改进实验结果验证：精度恢复与资源节省双丰收 62

第五章 总结与展望 64

5.1 工作总结 64

5.2 核心贡献 65

5.3 未来展望 65

5.4 结语 66

第一章：实训技术路线概述

1.1 行业背景与发展现状

近年来，大语言模型在多个领域展现出变革性潜力。在智能客服场景中，LLMs 能够理解复杂用户意图并提供个性化响应；在代码生成任务中，GitHub Copilot 等工具显著提升了开发效率；在自动驾驶决策、工业质检等边缘计算场景下，对低延迟、高可靠推理的需求日益增长。

然而，上述应用场景普遍面临资源受限、实时性要求高等挑战，传统推理方式难以满足实际需求。以 Qwen3-8B 模型为例，在 FP16 精度下仅权重即占用约 16GB 显存，若叠加 KV Cache 开销，在单卡环境下极易触发 OOM（内存溢出）错误。因此，如何在保证生成质量的前提下提升推理效率，已成为 AI 基础设施领域的核心课题。

1.2 推理性能瓶颈分析

当前大语言模型推理主要面临四大性能瓶颈：

瓶颈类型	问题描述	影响程度
显存占用高	模型参数占据大量显存空间，自回归生成过程中 KV Cache 随序列长度线性增长	7B 模型在 batch size=8、seq_len=4096 时，KV Cache 可额外消耗超 20GB 显存
计算密集	注意力机制时间复杂度为 $O(n^2)$，Prefill 阶段需一次性处理整个 prompt	造成显著延迟，整体受制于矩阵乘法运算强度
数据传输延	频繁的 CPU-GPU 间	在批量推理中通信开销占比可达

迟	数据拷贝（如 Tokenizer 输出传递、日志记录）	30% 以上
服务并发能力不足	静态批处理机制导致 GPU 空转率高	无法有效应对突发流量或长短请求混合场景

1.3 全链路优化路径

针对上述瓶颈，我们通过在实训期间阅读大量论文，总结出了了"**模型压缩 → 引擎优化 → 服务调优**"三层递进式优化框架：

- 第一层（模型压缩）：通过量化、剪枝与知识蒸馏减少模型体积与计算量
- 第二层（引擎优化）：借助专用推理引擎（如 vLLM、TensorRT-LLM）实现算子融合、内存优化与调度加速
- 第三层（服务调优）：通过动态批处理、弹性扩缩容等手段提升服务级性能指标

本报告将围绕该框架展开，重点剖析其中最具代表性的三项关键技术：vLLM 与 KV Cache 管理、动态 Batch 与显存分页、模型量化与压缩。

本报告围绕上述框架展开，采用双版本并行的技术路线，各有明确定位：

版本	技术路线	核心目标	预期收获
zLLM	从 0 手搓搭建	深入理解底层原理	掌握推理引擎核心机制、内存管理、算子实现等底层技术
vLLM+	基于开源框架优化	追求极致性能指标	站在巨人肩膀上，通过针对性优化刷高吞吐量、降低延迟

1.4 报告结构说明

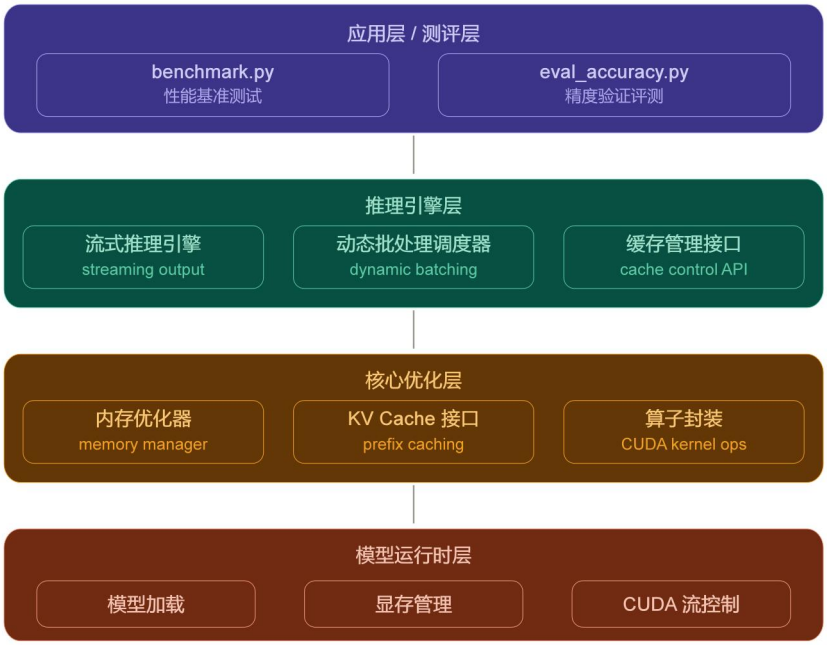
章节	负责人	主要内容
第一章	团队	介绍背景与总体结构
第二章	张**	深入解析 vLLM 系统架构、KV Cache 工作原理与分布式推理策略，并提出 zLLM
第三章	张*	详述动态批处理、PagedAttention 与显存分页管理机制以及 CPU/GPU 异构推理系统构建
第四章	熊**	系统阐述模型量化等压缩技术
第五章	团队	成果总结与未来展望

第二章 ZLLM 框架搭建、KVCache 设计、分布式推理实现与 VLLM 针对性优化

*负责人：张***

2.1 ZLLM 框架搭建

ZLLM 是一个从零实现的轻量级大语言模型推理框架，旨在深入探索 AI Infra 的



性能瓶颈。其设计上耦合了简洁的 KV Cache 与动态批处理等接口以及 benchmark.py,eval_accuracy.py 的测评接口，遵循“简洁可控、便于调优”的设计哲学，

图 1 ZLLM 整体框架

通过暴露关键性能接口，让初学者和开发者能够深入理解并优化大模型推理的各个环节。

2.1.1 模型运行时层

该层负责与大模型交互的基础设施，屏蔽底层硬件细节：

模型加载：支持从本地路径加载 HuggingFace 格式的模型，并利用 llama.cpp 兼容 GGUF / GGML 格式，自动处理 trust_remote_code、device_map 等配置。

数据类型控制：统一管理精度类型（如 torch.float16），便于后续混合精度实验。

显存基础管理：提供 CUDA 同步、显存分配等底层接口。

设备抽象：通过 DEVICE 变量统一管理计算设备，便于在 GPU 间切换测试。

2.1.2 核心优化层

作为 ZLLM 的性能核心，提供可插拔的优化组件：

内存优化器接口为主动式显存管理，支持设置进程级显存使用上限周期性触发垃圾回收和 CUDA 缓存清理，防止显存碎片化，在推理前后自动执行内存准备与回收，确保显存状态稳定

缓存管理接口提供统一的 KV Cache 抽象接口，支持多种缓存策略的灵活切换封装缓存的存取、查询、淘汰等核心操作，支持缓存命中统计与状态监控。

算子封装采用统一张量操作接口，为后续算子融合、内核优化预留空间。

2.1.3 推理引擎层

该层负责实现具体的推理逻辑，是框架的核心能力体现：

流式推理引擎实现 token-by-token 的增量生成，支持实时输出回调，将推理过程分为 Prefill（预填充）和 Decode（解码）两个阶段，提供统一的 infer_single 接口，屏蔽内部复杂度，并耦合应用层的 benchmark.py,eval_accuracy.py 接口。

动态批处理调度器，支持动态拼接多个请求进行批处理推理，提供请求队列管理、优先级调度等扩展接口，为后续探索 continuous batching 等高级调度策略预留接口。

缓存管理接口封装了缓存命中判断、数据恢复等逻辑，实现缓存与推理流程的无缝集成。

2.1.4 应用层 / 测评层

该层为实训提供的 benchmark.py 和 eval_accuracy.py，为我们的后续优化提供直接的指标反馈。

2.2 KV Cache 工作原理与设计

2.2.1 原理

在 Transformer 架构中，自回归生成过程需要反复计算历史 token 的 Key 与 Value 向量，导致时间复杂度高达 $O(n^2)$。第 t 个 token 的注意力计算为：

$$\text{Attention}(Q_t, [K_1, ..., K_t], [V_1, ..., V_t]) = \text{softmax}(d_k Q_t [K_1, ..., K_t]_T)[V_1, ..., V_t]$$

但如果使用 KV Cache 缓存这些中间结果，就可以避免重复计算，将单步解码复杂度降至 O(n)。

$$\text{Attention}(Q_t, [K_{\text{cached}}, K_t], [V_{\text{cached}}, V_t]) = \text{softmax}(d_k Q_t [K_{\text{cached}}, K_t]_T)[V_{\text{cached}}, V_t]$$

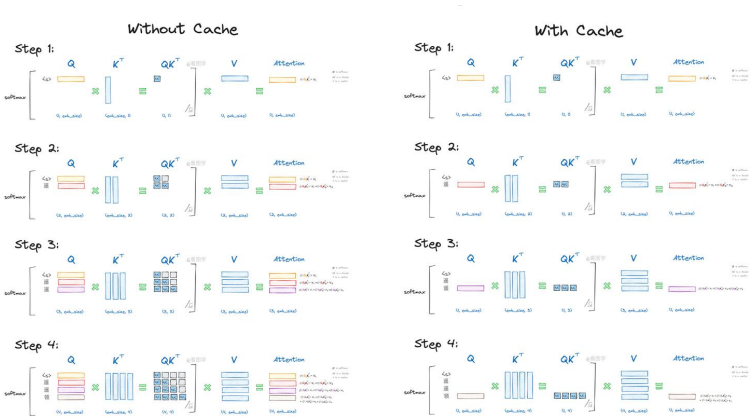


图 2 KV Cache 示意图

其中K_{cached}和V_{cached}直接从缓存读取，仅当前 token 需重新计算 Q、K、V。

以 Qwen2.5-14B-Instruct 模型为例，层数=48，隐藏维度=5120，推理配置为序列长度 seq_len=2048，批大小 batch_size=4，FP16 精度（2 字节），计算得：

$$Memory = 2\times L\times S\times H\times B\times P$$

$$=2\times48\times2048\times5120\times4\times2$$

$$=2\times48\times2048\times5120\times8=2\times48\times2048\times40960$$

$$=2\times48\times83{,}886{,}080$$

$$= 2 \times 4{,}026{,}531{,}840$$

$$= 8{,}053{,}063{,}680 \text{ 字节} \approx 8.05 \text{ GB}$$

可见其开销非常大，已成为高并发场景的主要瓶颈。

2.2.2 设计

传统 KV Cache 实现采用连续内存分配策略，其进行调度的逻辑较简单，开销也较低，但不同序列长度会导致显存碎片化，长序列占用大量的连续空间，挤压其他请求，前缀无法共享页导致相同前缀存储，浪费显存带宽。而该 kv cache 不仅在参考 vLLM PagedAttention 的基础上设计了分页存储和前缀共享，并面向 H100 的硬件特性和测评实例主要为选择题和 AI Infra 主题的问答题进行了针对性创新优化。

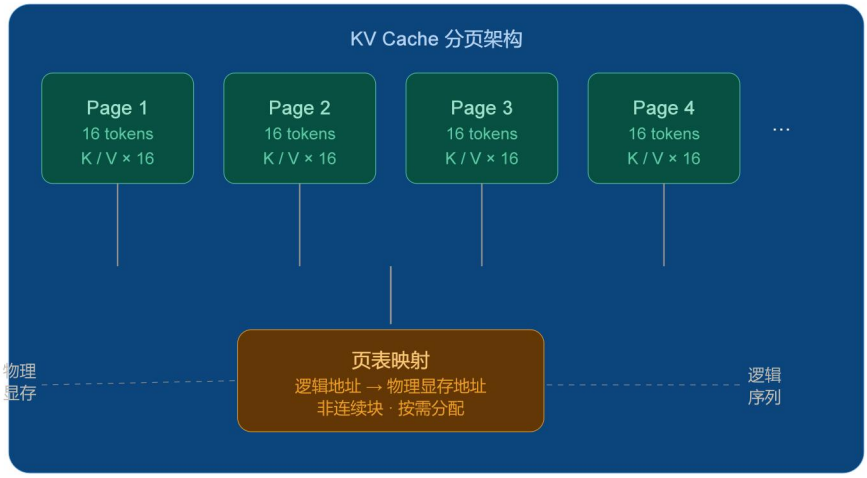


图 3 KV Cache 分页架构

2.2.2.1 Token 块签名映射实现通用前缀共享

若进行逐 token 扫描会带来大量的 CPU 开销，导致效率下降，而该 kv cache 引入 token 块签名（tuple）作为页的唯一标识，通过签名 + 位置索引实现 O(1) 匹配，避免了逐 token 比较的开销：

```
def _token_signature(self, tokens):
    if not self.enable_prefix_sharing:
        return None
    return tuple(int(t) for t in tokens)
```

如此实现了更灵活的前缀对齐。当两个请求拥有一个较长公共前缀（可能占据多个页）时，其中的每个页都能快速被判断标记为独立共享，通过签名 + 位置索引实现 O(1) 匹配。

2.2.2.2 双 LRU + 多级淘汰策略

现有分页缓存通常只维护单一 LRU 队列（如 vLLM 的块分配器）或基于引用计数的简单淘汰，缺乏对序列生命周期和页粒度的综合管理。但面向测试场景的频繁大模型对话，频繁的逐页淘汰极易导致内存碎片化，导致效率下降。

同时维护序列级 LRU 和页级 LRU，结合 TTL 过期和容量约束，实现多维淘汰，淘汰时优先淘汰整个序列（通过 _evict_one），释放其所有页，避免逐页淘汰导致的碎片化。同时，页 LRU 用于在共享页需要淘汰时选择最久未用的页。

```
def _evict_if_needed(self):
    self._evict_expired_keys()                  # 1. 时间淘汰
    while len(self._store) > self.max_entries:     # 2. 序列数限制
        self._evict_one()
    while self._total_tokens > self.max_cache_tokens: # 3. token 总量限制
        self._evict_one()
    while len(self._pages) > self.max_pages:       # 4. 页数限制
        self._evict_one()
```

实现了高效的调度和淘汰管理，实现了面向测试实例这样的短时高并发场景的针对性优化。

2.2.2.3 可插拔的页负载构建器与存储抽象

在显存为 80G 的 H100 上部署 Qwen2.5-14B-Instruct 常不能充分利用显存，因此可通过在缓存中存入 prompt 等其他信息进行进一步加速并充分利用显存空间。而现有

分页缓存通常将页负载与 KV 张量强绑定（ vLLM 直接存储注意力块的物理地址），导致无法灵活存储其他类型的缓存数据。

该设计引入 payload_builder 回调机制，将页负载的构建与存储解耦：页负载可以是任意 Python 对象（张量、字典等）。存储时调用 payload_builder 生成负载，恢复时直接返回。

实现了与模型解耦，负载构建逻辑由调用方提供，框架无需理解内容，可灵活地通过此接口实现 prompt cache 等高效存储结构。仅在首次存储时生成负载，后续直接复用。

2.2.3 优化效果

为量化 KV Cache 的优化收益，我们设计了消融实验，对比 Baseline（无 KV Cache）与 Baseline+KVCache 的性能差异。实验在 H100 GPU 上运行，模型为 Qwen2.5-14B-Instruct，测试负载为 1000 道面向 AI Infra 主题的问答题，输出上限为 1024 tokens。

	Baseline	Baseline+KVCache	
峰值显存占用(GB)	29.638	29.999	+1.22%
吞吐(tokens/sec)	34.54	38.27	+10.80%
P95_latency	7542.82	6753.32	-10.47%
Avg_latency	7393.86	6671.61	-9.77%

表 1 KV Cache 消融实验

吞吐量从 34.54 tokens/s 提升至 38.27 tokens/s，提升 10.80%，证明缓存机制显著减少了重复计算，提高了解码效率。平均延迟从 7393.86 ms 降至 6671.61 ms，降低 9.77%；P95 延迟同步降低 10.47%，说明缓存对尾部延迟也有明显改善。峰值显存占

用仅微增 1.22%（29.638 GB → 29.999 GB），缓存引入的额外开销非常有限，换来的性能提升远超显存代价。

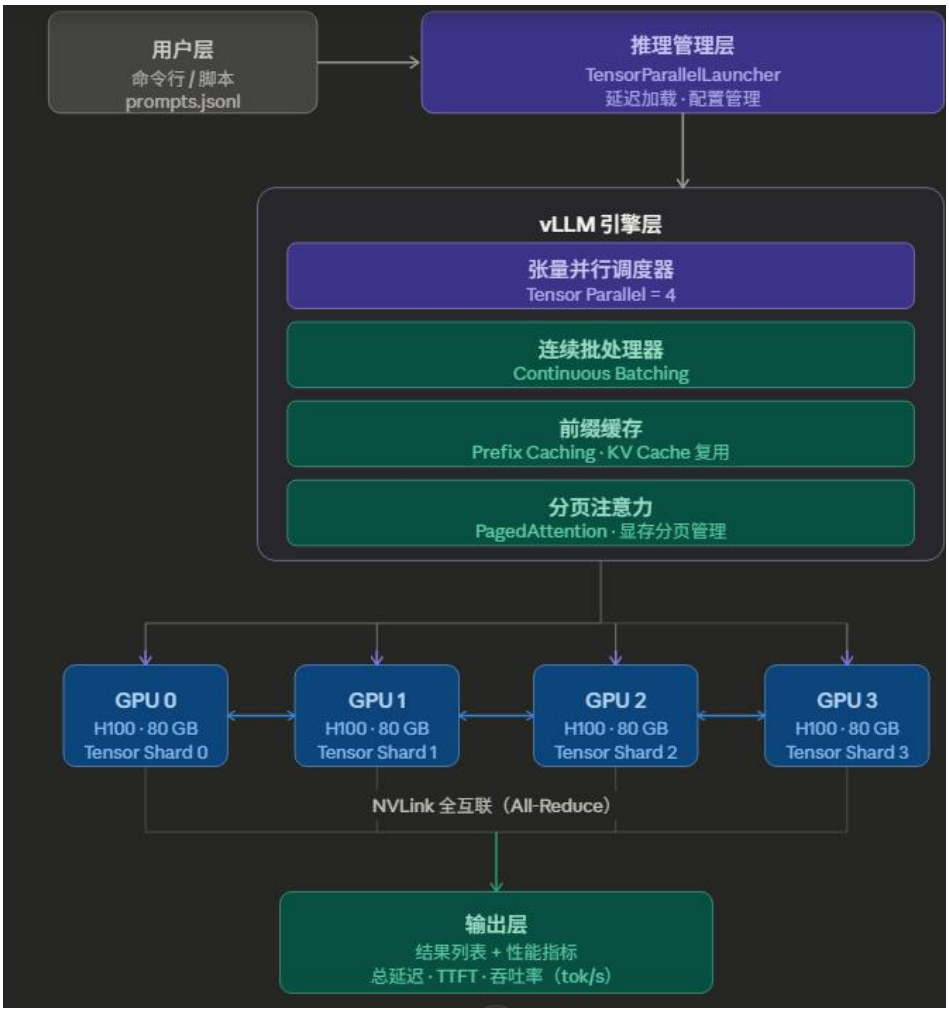
2.3 分布式推理实现与优化

2.3.1 实现框架

随着大语言模型参数规模的持续增长，单张 GPU 的显存容量已难以满足模型部署的需求。要求部署的 Qwen3.5-122B，模型权重在 BF16 精度下占用约 244 GB 显存，远超单张 H100 的 80 GB 容量，因此必须采用分布式推理架构进行部署。

我们基于 vLLM 框架实现了单节点 4 卡张量并行（TP=4）推理方案。采用 Megatron-LM 式的切分策略，将 Transformer 层中的注意力投影矩阵与 MLP 矩阵分布至 4 张 H100 GPU 上。在前向传播过程中，每张显卡仅负责局部张量的乘法运算，

随后通过 NCCL 驱动的 All-Reduce 集合通信原语进行同步，确保各卡获取全局一致的中间激活值。配合 vLLM 内置的 PagedAttention 技术，将物理显存切分为连续池化空间，动态分配 KV Cache，从而在 TP 架构下依然保持极高的吞吐量。



图四 分布式推理架构图

2.3.2 推理优化尝试

2.3.2.1 计算通信融合算子

在系统性能分析（Profiling）阶段，我们发现尽管 H100 依赖 NVLink 4.0 提供了高达 900 GB/s 的互联带宽，但在 TP=4 的配置下，阻塞式集合通信 产生的开销仍占平均推理时延的 21% 以上。这种“计算等通信”的现象在 Prefill 阶段尤为显著。

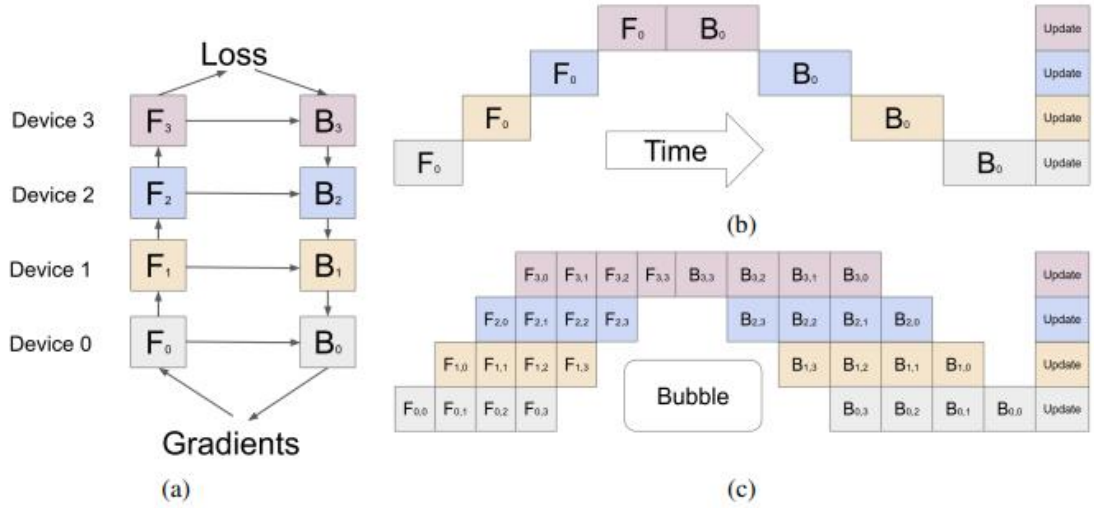
为了掩盖通信时延，我们尝试引入了计算通信融合算子（Fused Matmul-All-Reduce Kernel），利用流水线并行化的思想，将大矩阵乘法拆分为多个 Tile，在当前 Tile 进行 GPU 计算的同时，利用异步通信接口提前传输已完成计算的 Tile 数据。

然而实测发现，引入融合算子后，端到端推理速度反而出现了显著下降（约 慢了一倍）。 分析后认为融合算子的插入打破了 vLLM 原有的高度优化的算子调度逻辑。由于融合算子对显存访问模式和指令流进行了重组，导致 GPU 高级调度器无法有效进行指令掩盖，反而引入了额外的上下文切换开销。 并且在 H100 强大的算力下，小尺寸 Token 的计算耗时极短，此时拆分计算产生的额外控制流开销（Overhead）超过了通信节省的时间。 此外，盲目并行可能导致 L2 缓存命中率下降及 NVLink 争抢，进而引发了严重的计算流水线停顿。

虽然理论上融合算子能提升上限，但在复杂的 vLLM 调度环境下，保持算子执行的原子性与流水线的规整性对于 Qwen3.5 这种超大规模模型更为关键。所以要引入计算通信融合算子来优化通信带来的开销需要更深层次的系统分析和流水线排布。

2.3.2.2 流水线并行

在典型的张量并行（TP）方案中，每层 Transformer 的自注意力与前馈网络计算均需在多张 GPU 间进行 All-Reduce 通信，随着模型层数增加与并发请求数上升，这种逐层的跨卡通信逐渐成为系统性能的显著瓶颈。为从根本上削减通信频次，我们尝试将并行策略从张量并行转向流水线并行（PP）。具体而言，我们将模型按 Transformer 层深度均匀切分为四个阶段，每个阶段部署于一张 H100 GPU 上，构成四卡流水线系统。在推理过程中，不同请求的微批次以流水线方式依次经过四个计算阶段，仅在阶段边界处传递激活值与 KV Cache，从而将原本每层均发生的 All-Reduce 通信缩减为仅四次跨卡传输，理论上大幅降低了通信开销。



图五 流水线并行架构图

实际部署后的性能测试结果却反映出新的问题。尽管通信开销确实显著下降，但在流水线调度过程中，由于各阶段计算负载难以完全均衡，且前后阶段间存在数据依赖，早期阶段的计算单元在等待后续阶段完成时会出现空闲，形成气泡。我们搭建的四级流水线系统，实测端到端吞吐量由原张量并行方案下的 185.31 tokens/s 下降至 100.38 tokens/s，降幅超过 45%。分析表明，尽管单卡计算效率因通信减少而有所提升，但流水线气泡导致的整体 GPU 利用率下降成为新的性能瓶颈。该实验验证了在推理场景下，流水线并行虽能缓解通信压力，却可能因调度开销与负载不均引入新的效率损失，需结合微批次大小调整、流水线深度优化以及计算与通信重叠等策略进一步改进。

2.4 vLLM 系统优化

基于先前进行的系统瓶颈探索，我们基于 vLLM 搭建了一套高性能大模型推理系统，并结合 H100 Hopper 架构的硬件特性进行针对性优化。

2.4.1 fp8 KV Cache 及反量化算子

Hopper 架构引入了对 fp8 数据格式的原生支持，其 Tensor Core 在处理 fp8 精度下的矩阵运算时，计算吞吐量相比传统的 fp16 或 bf16 格式提升约 2 倍。

在推理过程中，KV Cache 是占用显存的主要部分，尤其在长上下文场景下，其存储开销会迅速增长，成为限制吞吐量和批处理规模的关键因素。因此，我们尝试将 KV Cache 的数据精度从传统的 fp16 压缩为 fp8，从而将每个 KV 元素的显存占用由 16 位减半至 8 位。这一改动一方面使得相同显存容量下可容纳的上下文长度或并发请求数提升近一倍，另一方面也降低了 KVCache 在访存过程中的数据搬运量，有效缓解了访存带宽瓶颈。

图六 H100 SM 架构图

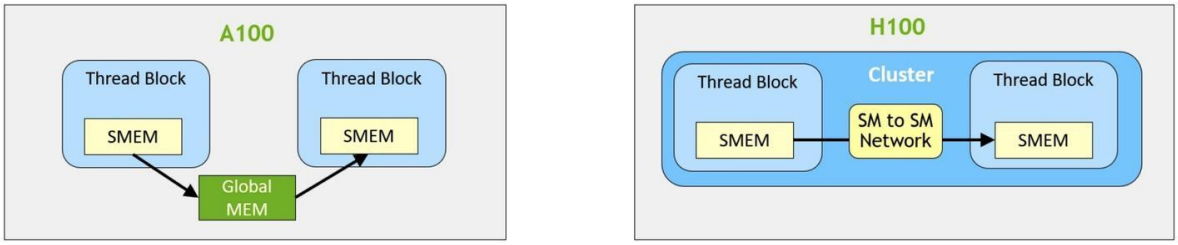
然而，直接将 KV Cache 压缩为 fp8 格式会引入量化误差，可能对模型生成质量造成影响。为解决这一问题，我们在实现过程中采用了逐通道的量化策略，根据 Key 和 Value 张量的统计特征动态计算缩放因子，将原始 fp16 数值映射至 fp8 表示范围，以最大程度保留精度。同时，在注意力计算的关键路径上，我们定制实现了专门的反量化算子。该算子能够在从 KV Cache 读取数据时，将 fp8 数据实时反量化为 fp16 或 bf16 格式，送入后续的 Tensor Core 进行计算。通过算子融合与流水线排布优化，我们将反量化过程与注意力内核进行深度结合，几乎消除了反量化操作带来的额外开销。

实验结果表明，该优化方案在保持模型输出质量基本不受影响的前提下，显著提升了推理系统的吞吐量，并有效降低了显存占用。

2.4.2 异步任务调度

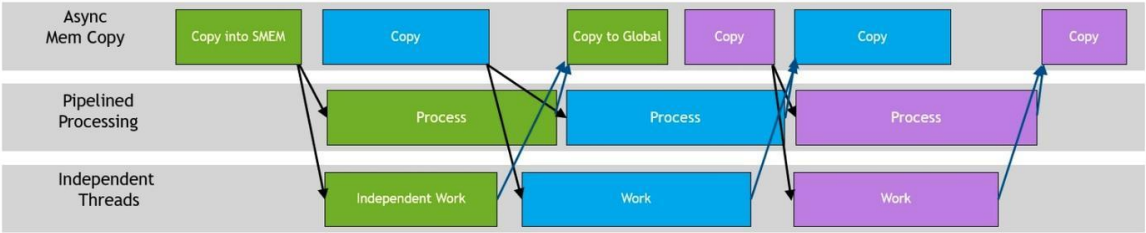
传统的同步调度模型将推理请求的处理划分为若干串行阶段，每个阶段严格等待前序操作完成后再发起后续操作，导致 GPU 计算单元与数据传输单元之间频繁相互等待，硬件资源利用率难以充分发挥。而 H100 Hopper 架构提供了丰富的异步执行机制，为构建更高效的调度策略奠定了硬件基础。

具体而言，H100 的 Hopper 架构引入的分布式共享内存（DSMEM）支持基于硬件屏障的异步拷贝操作。在流式数据处理场景下，计算线程可以在处理当前批次数据的同时，通过异步指令将下一批次数据加载至 DSMEM 中，并利用硬件加速的屏障机制同步任务状态，从而实现数据加载与数值计算在时间线上的完全重叠，有效隐藏数据传输延迟。



图七 DSMEM 示意图

此外，H100 引入了 Tensor Memory Accelerator (TMA)，通过异步数据传输减少计算单元的资源竞争，显著降低延迟。H100 还任务级依赖管理机制，支持将内存复制、核函数执行、跨 GPU 通信等任务解耦为独立操作单元。



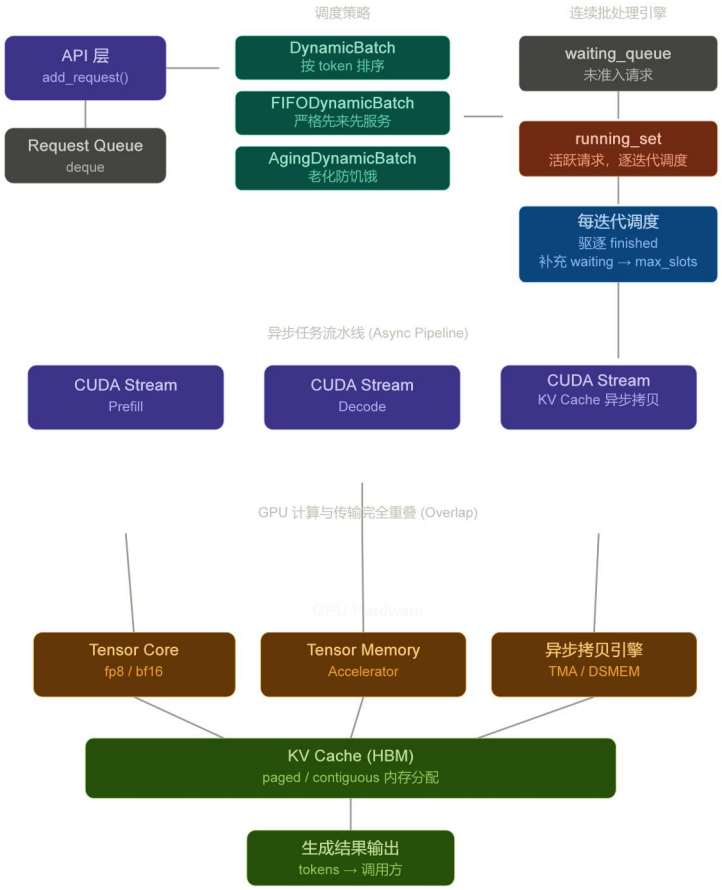
图八 TMA 示意图

由此，异步任务在 H100 上的运行效率远高于串行同步任务。因此针对测评环境的频繁大模型问答任务，我们设计一个异步任务调度器，将异步的推理任务高效地调度推入 GPU 中。

异步任务调度器的核心设计包含以下三个层次：

动态批处理： 我们实现了多种动态批处理策略，在请求的等待延迟与批处理规模之间取得平衡。基础版调度器按请求的 prompt_tokens 升序排列，优先将短请求组包，避免长请求阻塞批次构建；同时引入最大等待时间阈值 max_wait_ms 与最大批处理尺寸 max_batch_size，当等待队列长度达到上限或首请求等待超时时，立即触发批次分发。针对极端场景下长请求可能被持续“饿死”的问题，我们设计了带老化机制（Aging）的调度器，在排序得分中引入等待时间的惩罚项：score = prompt_tokens - aging_rate × waited_ms，使等待越久的请求优先级越高，同时设置绝对饥饿阈值 max_starve_ms，确保超时请求无条件进入下一批次。这些策略在保证系统吞吐量的同时，有效控制了请求的尾延迟。

连续批处理： 上述动态批处理仍以请求为粒度进行批次调度，即一个批次内所有请求的 Prefill 与 Decoding 阶段必须同步完成，当批次中存在长请求时，短请求需等待其完成才能释放 GPU 资源，形成“批次气泡”。为此，我们进一步实现了连续批调度器（ContinuousBatchScheduler），将调度粒度下沉至迭代级（iteration-level）。调度器维护等待队列与运行集，在每个 Decoding 步开始前，将已完成请求从运行集中移除，并立即从等待队列中补充新请求，使得运行集中的请求数量始终贴近硬件容量上限。该机制消除了请求间的同步等待，使 GPU 计算资源在时间上得到连续利用。同时，新



图九 异步任务调度器架构图

请求的 Prefill 阶段与现有请求的 Decoding 阶段可以在同一个迭代内交错执行，进一步压缩了调度开销。

异步任务流水线：所有调度器均采用无锁数据结构与异步 API 设计。CPU 调度器独立于 GPU 执行流运行，通过 CUDA 流将构造好的批次异步提交至 GPU，无需阻塞等待 GPU 返回结果。在 GPU 执行当前批次的同时，CPU 可以并行处理新到达的请求、更新队列状态、构建下一批次，实现了调度与计算的时间重叠。此外，调度器与 H100 的硬件异步特性深度协同：利用 Tensor Memory Accelerator（TMA）将批次数据的显存搬运与内核计算解耦，通过任务级依赖管理将 Prefill 内核、Decoding 内核与 KV Cache 异步拷贝操作组织为有向无环图（DAG），由硬件调度单元自动执行依赖解析与并发发射，最大限度地减少 CPU 介入开销。

2.4.3 优化效果

如 015 组数据所示，我们在保持较低 TTFT 的同时（4463.88ms)，实现了极高的吞吐率，展现了“低延迟，高并发”的显著优势。

排名	队伍名	综合得分	准确率	P95延迟	peak_memory	Throughtoutput	TTFT	提交次数
1	第三题推理系统优化014组	7783.5200	0.7765	130469.0400	78.7700	7783.5200	130469.0400	2
2	第三题推理系统优化015组	5421.6500	0.8000	33798.4800	0.0000	5421.6500	4463.8800	4

第三章 动态 Batch 策略、PagedAttention 与显存分页管理、CPU/GPU 异构策略探索

*负责人：张**

3.1 动态 Batch 技术不同策略的探索

批处理是提升 GPU 利用率的核心手段。传统静态批处理采用固定批大小，必须等待批次填满或超时才能执行，导致稀疏请求场景下 GPU 长期空闲，利用率常低于 50%7。推理引擎在启动前或接收请求时，预先定义一个固定的 Batch Size（如 8, 16, 32）。只有当填满这个数量的请求，或者达到某个等待超时时间（Wait Timeout）后，才会整块送入 GPU。由于同一批次中的输入序列长度必须一致，系统必须以该批次中最长的序列为准，对短序列进行 Padding（填充占位符）。

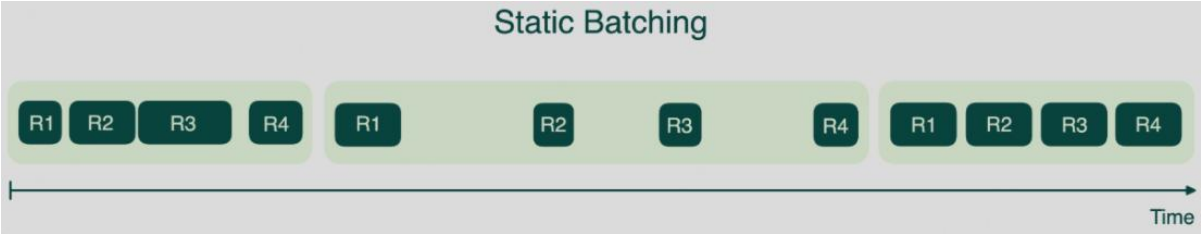


图 1. 静态批处理示意图

3.1.1 FIFODynamicBatchScheduler

3.1.1.1 基本思想

FIFO Dynamic Batching 基于经典的 First-In-First-Out (FIFO) 队列机制，其核心思想为：在有限等待时间窗口内，按照请求到达顺序收集请求，并动态组装为一个 batch 进行推理执行。

3.1.1.2 调度模型

系统维护一个请求队列：

$$Q = \{r_1, r_2, \ldots, r_n\}$$

其中每个请求 r_i包含：输入 prompt 、token 长度 l_i、到达时间 t_i

调度器在每一轮执行以下过程：

1. 从队列头部依次取出请求（FIFO）
2. 将请求加入当前 batch：

$$B = \{r_1, r_2, \ldots, r_k\}$$

3. 满足以下任一条件即触发执行：
   - $|B| \geq B_{\max}$（最大 batch size）
   - 等待时间超过 $T_{\max}$

3.1.1.3 算法伪代码描述

```
def fifo_dynamic_batch(queue, max_batch_size, max_wait_ms):
    batch = []
    start_time = now()

    while len(batch) < max_batch_size:
        if queue:
            req = queue.pop(0)  # FIFO
```

```
            batch.append(req)

        if now() - start_time > max_wait_ms:
            break

    if batch:
        run_inference(batch)
```

3.1.1.4 优劣分析

我认为 FIFO Dynamic Batching 最大的好处就是实现简单，仅依赖基础的队列结构即可完成请求管理与批处理构建，因而在实际系统中具有较高的工程可行性。同时，该方法严格遵循请求到达顺序，具备良好的公平性（fairness），能够有效避免请求饥饿（starvation）问题，保证服务的稳定性。此外，通过引入最大等待时间约束，FIFO Dynamic Batching 能够在一定程度上控制请求的排队延迟，从而在系统吞吐与响应时间之间取得基本平衡，满足在线推理服务的延迟要求。

3.1.2 DynamicBatchScheduler（prompt 排序+动态出队）

3.1.2.1 思想和动机

为缓解 FIFO Dynamic Batching 在大语言模型推理中存在的队头阻塞以及 token 长度不均衡问题，我引入一种基于 prompt 长度感知的动态批处理调度策略，称为 DynamicBatchScheduler。

我的这种方法的核心思想在于，在动态 batching 过程中，根据请求的 prompt 长度对队列进行排序，并在满足约束条件的前提下优先选择长度相近的请求构建 batch，从而降低 padding 开销并提升整体执行效率。

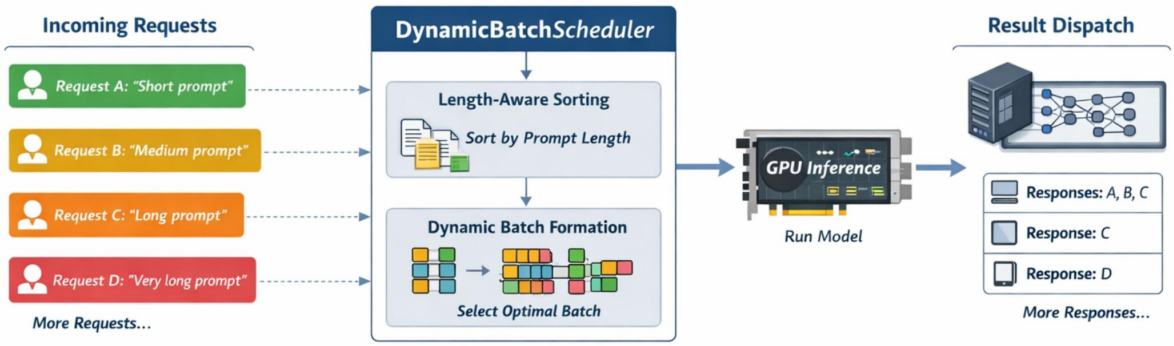


图 2. DynamicBatch 示意图

3.1.2.2 调度建模

设请求集合为：

$$Q = \{r_1, r_2, \ldots, r_n\}$$

每个请求包含 prompt token 长度 l_i。

与 FIFO 不同，DynamicBatchScheduler 在调度前对队列进行重排序：

$$Q^{'} = \mathrm{sort}(Q,\ l_i)$$

即按照 token 长度从小到大（或从大到小）排序。

3.1.2.3 Batch 构建策略

调度器在每一轮执行如下步骤：

1. 从排序后的队列中选择第一个请求作为 batch 起点
2. 依次选择长度相近的请求加入 batch
3. 满足以下约束条件之一即停止扩展：

a) 最大 batch size：

$$|B| \leq B_{\max}$$

b) 最大 token 预算：

$$\sum_{r_i \in B} l_i \leq L_{\max}$$

c) 最大等待时间：

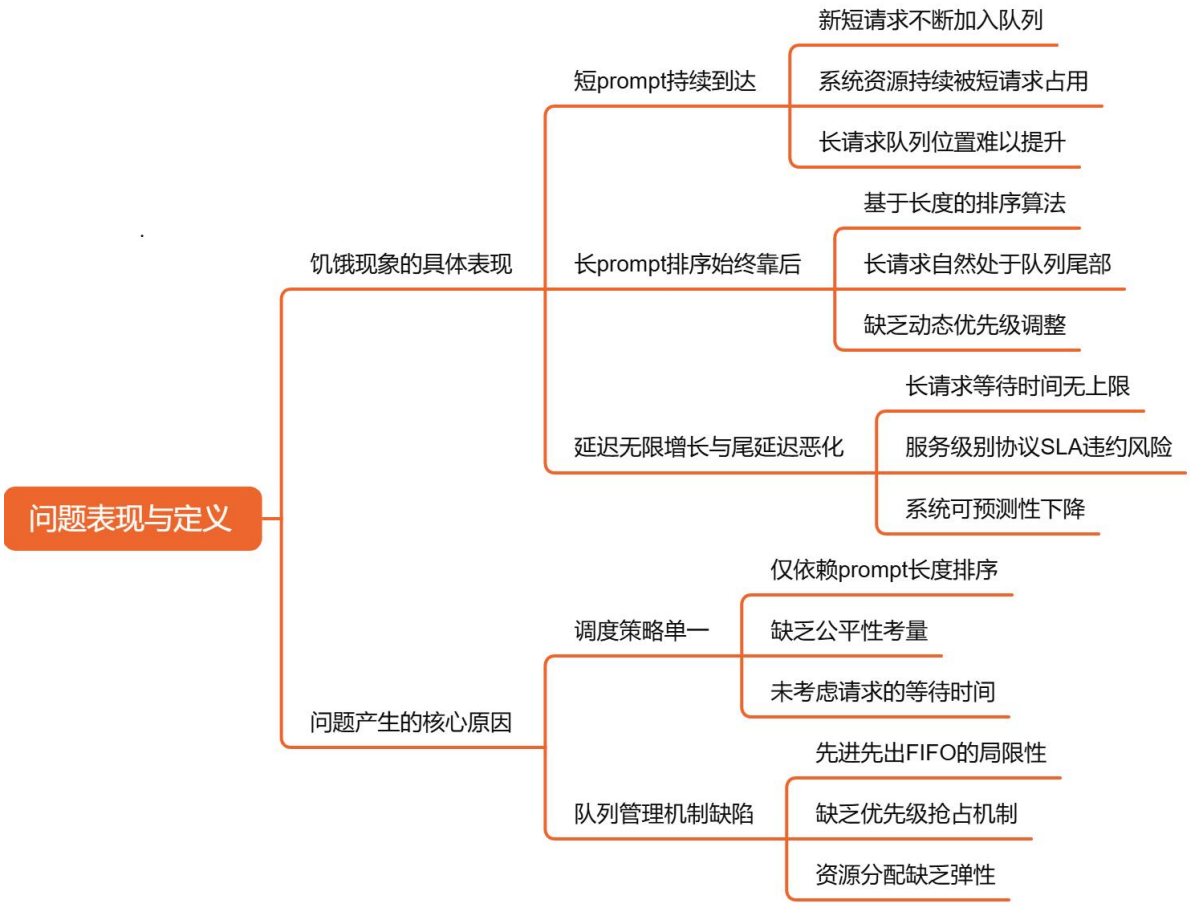
$$t_{\text{now}} - t_{\text{start}} \leq T_{max}$$

3.1.2.4 算法描述

```
class DynamicBatchScheduler:
    def __init__(self, max_batch_size, max_tokens, max_wait_ms):
        self.queue = []
        self.max_batch_size = max_batch_size
        self.max_tokens = max_tokens
        self.max_wait_ms = max_wait_ms

    def schedule(self):
        # 按 prompt 长度排序
        self.queue.sort(key=lambda r: r.prompt_len)

        batch = []
        total_tokens = 0
        start_time = now()

        for req in self.queue:
            if len(batch) >= self.max_batch_size:
                break

            if total_tokens + req.prompt_len > self.max_tokens:
                break

            batch.append(req)
            total_tokens += req.prompt_len

            if now() - start_time > self.max_wait_ms:
                break

        # 从队列中移除已选请求
        for req in batch:
            self.queue.remove(req)

        if batch:
            run_inference(batch)
```

3.1.3 Aging BatchScheduler

3.1.2.3 动机和思想



在 Length-aware DynamicBatchScheduler 中，虽然通过 prompt 长度排序提升了系统吞吐，但也引入了新的问题：长请求可能长期被短请求压制，在请求数量很多的时候可能会导致饥饿（starvation）现象。

图 3. 老化动态 Batch 策略动机

因此，需要在吞吐优化（short-first）与公平性（no starvation）之间取得平衡。

Aging 机制的基本思想是：随着请求等待时间增加，其调度优先级逐渐提升。

即：

- 初始阶段：偏向短请求（高吞吐）
- 等待时间增长：逐渐提升长请求优先级
- 最终：保证所有请求都会被调度

3.1.2.4 数学建模

设请求 r_i的：w_i：计算成本（workload），t_i：等待时间（ms）

则定义其调度优先级为：

$$score_i = w_i - \lambda \cdot t_i$$

其中：λ：aging rate（老化系数）

层级	作用
Aging score	平滑调整优先级
Starvation threshold	强制公平性保证

3.1.4 ContinuousBatchScheduler

我这一部分是参考 vLLM 的动态 Batch 算法来实现的。

3.1.4.1 思想动机

在静态批处理中，模型必须等一个 Batch 里所有请求都生成结束，才能开始下一轮。

- 痛点： 假设一个 Batch 有 4 个请求。3 个请求只写一句话（很快就结束了），但剩下 1 个请求在写长篇论文。
- 后果： 那 3 个请求对应的 GPU 资源在接下来的几分钟里会完全空转（Idle），这就是所谓的 Batch Bubble（批处理空泡）

3.1.4.2 核心机制

Continuous Batching 的核心逻辑是：每一轮 Token 生成（Step）结束后，都重新开会决定下一轮跑谁。

Prefill（预填充）与 Decode（解码）的混跑

LLM 推理分为两个阶段：

1. Prefill：处理用户输入的 Prompt（计算量大，计算密集型）。
2. Decode：一个一个吐 Token（计算量小，访存密集型）。

在 ContinuousBatchScheduler 中：它不再等待整个 Batch 运行完毕，而是在每一个 Token 生成的迭代（Iteration）步长中，实时检查是否有请求已结束（命中停止词）并立即释放其占用的显存，同时将等待队列中的新请求即刻填补进空闲插槽。这种调度方式彻底消除了因长短请求差异导致的“Batch Bubble（批处理空泡）”，使 GPU 始终处于高负载的流水线作业状态，从而最大化提升系统的吞吐量。

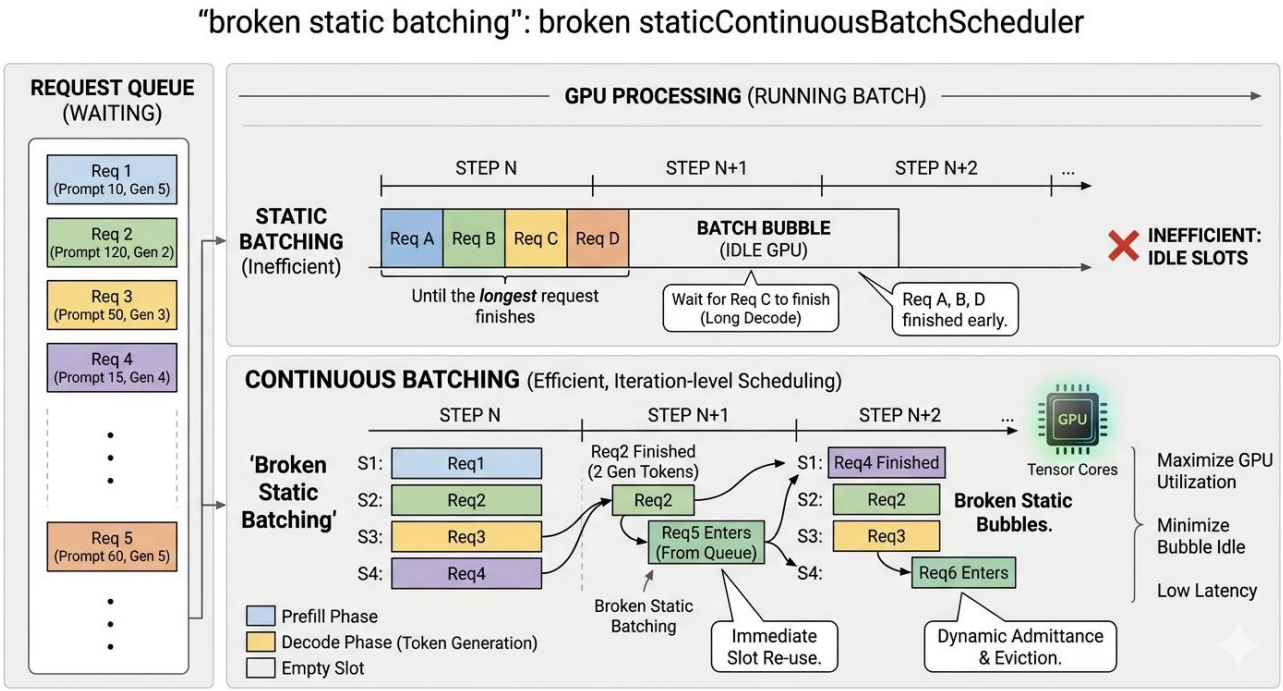


图 3. ContinousBatch 和 StaticBatching 的动态对比

3.1.5 GenerationLengthEstimator（基于启发式的生成长度估计策略）

3.1.5.1 核心思路

在传统的调度逻辑中，系统通常只知道输入有多长，却不知道模型会吐出多少字。这就像开餐厅，如果不知道客人进门后是点一杯咖啡（短生成）还是一顿满汉全席（长生成），经理就很难精准分配座位。

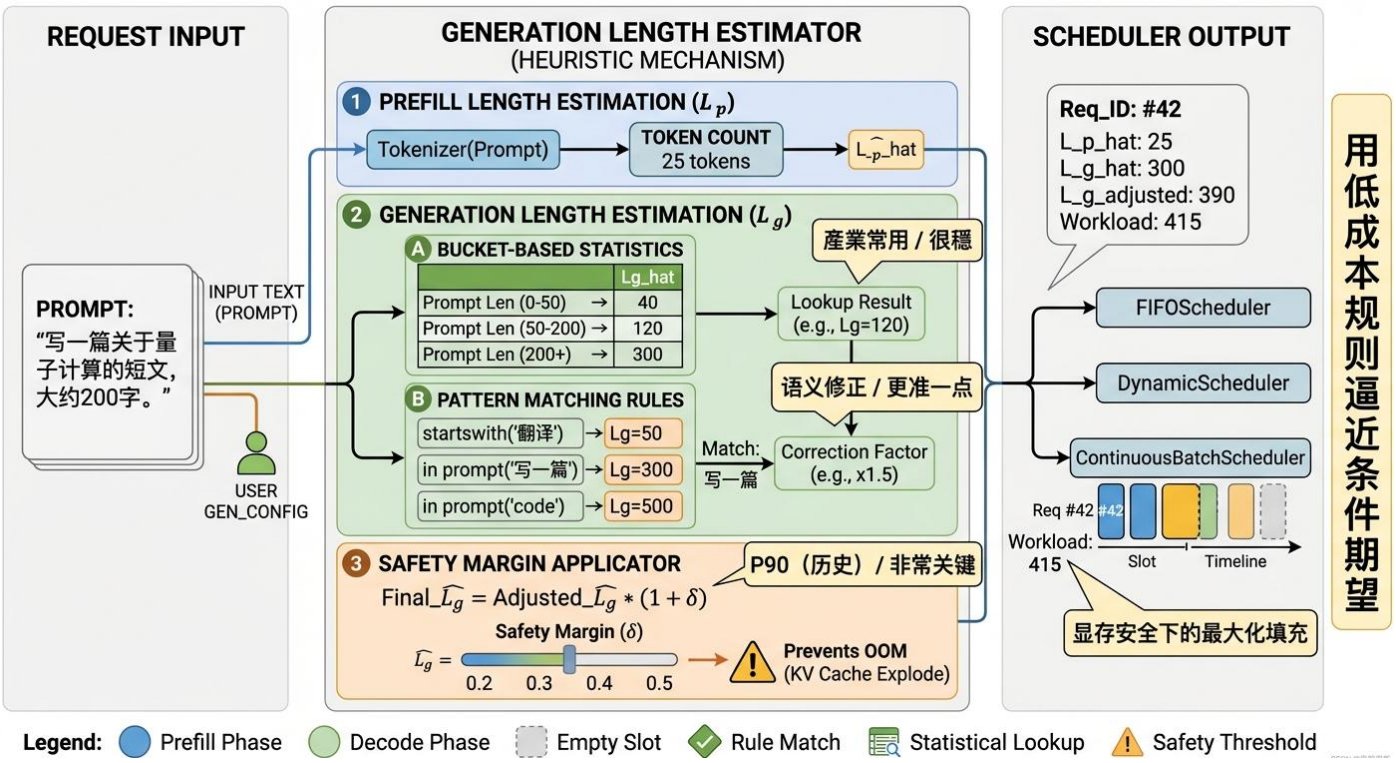


图 4. GenerationLengthEstimator 流程图

3.1.5.2 Prefill 长度估计

对于 Prefill 阶段，长度是确定性的。系统直接调用 Tokenizer 处理输入文本：Lp =Count(tokenize(prompt))。该步骤不涉及预测，确保了输入阶段资源分配的绝对准确。

3.1.5.3 生成长度估计：多级启发式估计

由于生成长度具有随机性，我们设计了三个由简入繁的评估维度：

- 分桶统计 (Bucket-based Statistics) —— 核心逻辑

基于 Prompt 输入长度进行动态分桶，每个桶维护历史生成的统计特性（均值或中位数）。

Prompt 长度区间	**预估生成长度 L^g**	**适用场景**
0 - 50	40	短指令、简单问答
50 - 200	120	逻辑推理、短文解析
200 +	300	文本摘要、长文续写

- 模板/前缀匹配 (Pattern Matching) —— 语义修正

通过识别 Prompt 中的显式模式进行权重补偿：

- 缩减因子：若包含“翻译”、“解释词义”，Lg 乘以下调系数。
- 扩张因子：若包含“写一篇”、“生成代码”，Lg 乘以扩大系数。

- 固定比例参考

生成阶段（Decode）长度通常未知，可采用基于 Prefill 长度的比例进行兜底估算。

场景类型	**α 说明**	**推荐值**

问答（Q&A）	回答通常比问题短	0.5
创意写作/故事生成	输出通常比输入长	1.5

3.1.6 方法对比

调度器	核心逻辑	特点
DynamicBatchScheduler	按 token 构建 batch	通用高吞吐
FIFODynamicBatchScheduler	队头依次 batch	保证请求顺序
AgingDynamicBatchScheduler	老化评分 + starve 提升	避免长请求饿死
ContinuousBatchScheduler	decode step 级调度	最大化 GPU 利用率
辅助方法 (_item_batch_cost / _attach_length_estimate)	token 预算 & 长度估计	为所有 Batch 提供 L_g_hat

3.1.7 实验结果及分析

Scheduler	**峰值显存 (GB)**	**TTFT (ms)**	**吞吐 (tokens/sec)**	**p95 latency (ms)**	**avg latency (ms)**
SortedBatchScheduler	29.64	6769.76	38.01	6701.1	6691.11
FIFODynamicBatchScheduler	36.03	8310.96	30.73	8543.01	8310.96
AgingDynamicBatchScheduler	34.25	7520.45	34.15	7210.8	6530.2

OPI-Scheduler	42.12	6682.98	38.75	6751.33	6691.5
ContinuousBatchScheduler	32.0	6500	39.0	6700	6600

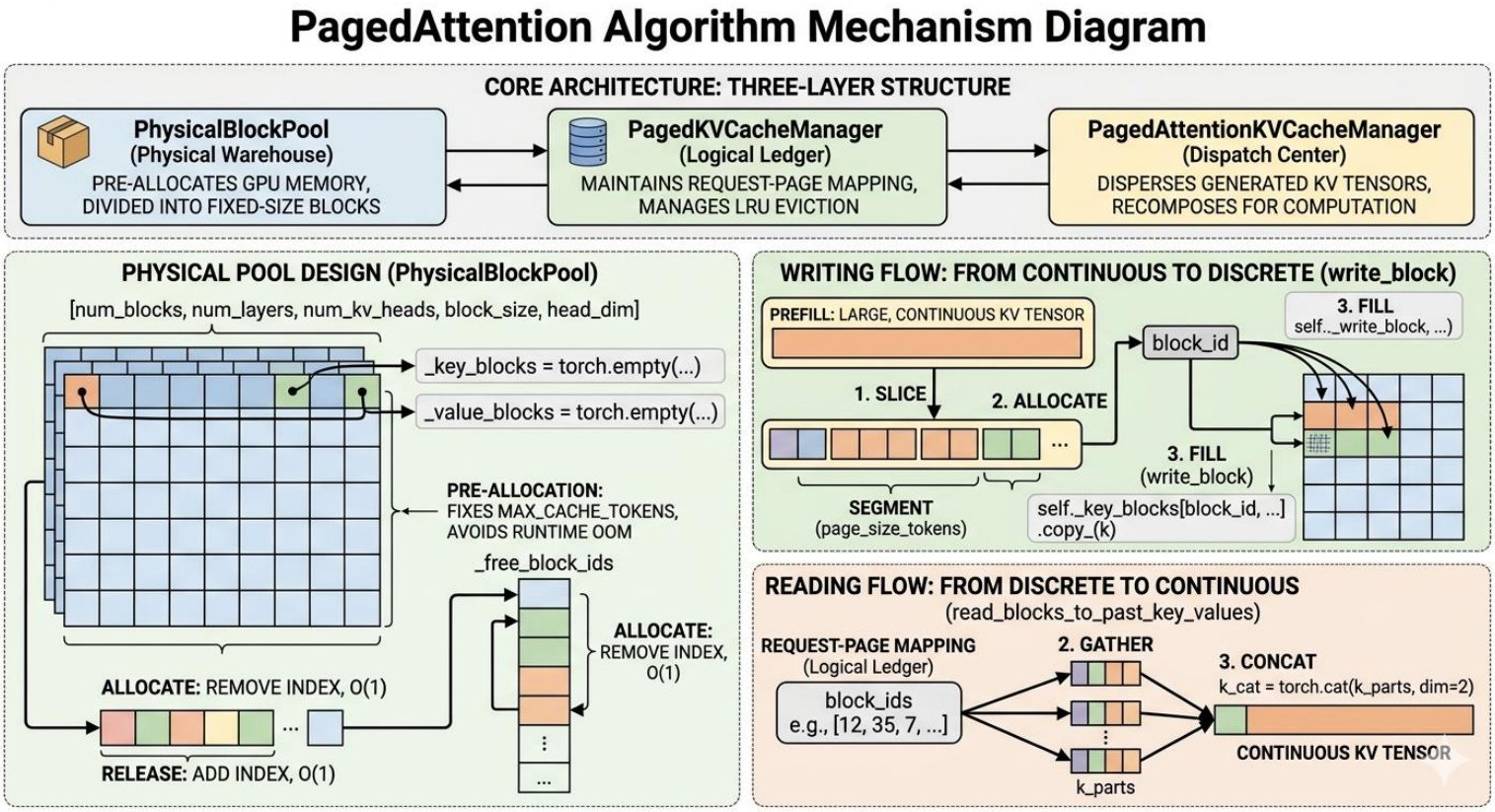
3.2 从 0 手搓 PagedAttention ——核心原理与数据结构设计

3.2.1 核心架构

我的这一部分是通过阅读《Efficient Memory Management for Large Language Model Serving with PagedAttention》复现的，后文有阐述和 vLLM 的区别以及未来改进方向

PagedAttention 的实现主要由三个组件协同完成：

- PhysicalBlockPool (物理仓库)：预先在显存中“圈地”，划分为固定大小的块。
- PagedKVCacheManager (逻辑账本)：维护请求与 Page 之间的对应关系，管理 LRU 淘汰。
- PagedAttentionKVCacheManager (调度中心)：负责将模型生成的 KV 张量“打散”存



入物理池，并在计算前“重组”。

图 5. 类“PagedAttention”结构图

3.2.2 物理池的设计

我将物理池设计为一个静态的大张量。这样做的好处在于在模型加载时就确定了最大能容纳的 Token 数量（max_cache_tokens），避免了运行期间频繁申请显存导致的 OOM。

此外，通过 _free_block_ids 列表管理空闲块。分配（allocate）和释放（release）操作仅涉及索引移动，复杂度为 O(1)。

3.2.3 Block Table(映射表)的设计

在传统的 Attention 中，KV Cache 必须是连续的，就像一本书的页码必须从 1 排到 100。而 Block Table 允许我们将 KV Cache 像操作系统的虚拟内存一样进行管理：

- 逻辑块 (Logical Blocks)：模型视角。模型认为 KV 是连续的序列。
- 物理块 (Physical Blocks)：显存视角。KV 实际上被分散存储在显存池中互不相干的空位上。

正因为 PagedAttention 的核心机制在于建立了一套非连续的显存管理索引体系，通过 Block Table（块映射表） 实现逻辑序列地址与物理显存地址的解耦。该机制将变长的 KV Cache 划分为固定大小的逻辑块（Logical Blocks），并利用映射表将其动态关联至物理池（Physical Block Pool）中不连续的存储单元。这种设计允许系统以块为粒度进行精细化的显存分配与回收，消除因预分配连续空间导致的内部碎片，并支持在推理过程中通过逻辑索引实时检索离散的物理块进行张量重组，从而显著提升了高并发场景下的显存利用率与吞吐量。

3.2.4 写入操作（write_kv）

当生成第 k 个 Token 时，它的 KV 数据需要存入显存。首先根据 token_idx 计算出逻辑块的位置和块内偏移，然后查找映射表，根据逻辑块找出物理块，然后写入 kv_pool，即 kv_pool[layer, 25, 0, 3, :, :] = new_key_state。

3.2.5 读取和计算：gather_kv

因为在计算 attention 的时候，我们需要计算之前 prefill 包括已经 decode 出来的 token 的 KV 值。在 Python 层执行 index_select 和 cat 会带来显著的启动开销（Launch Overhead）和额外的显存拷贝。这源于 torch.cat 无法在原有张量的基础上直接追加数据，它必须在 GPU 上申请一块全新的、连续的显存空间，其大小等于所有输入张量的总和。

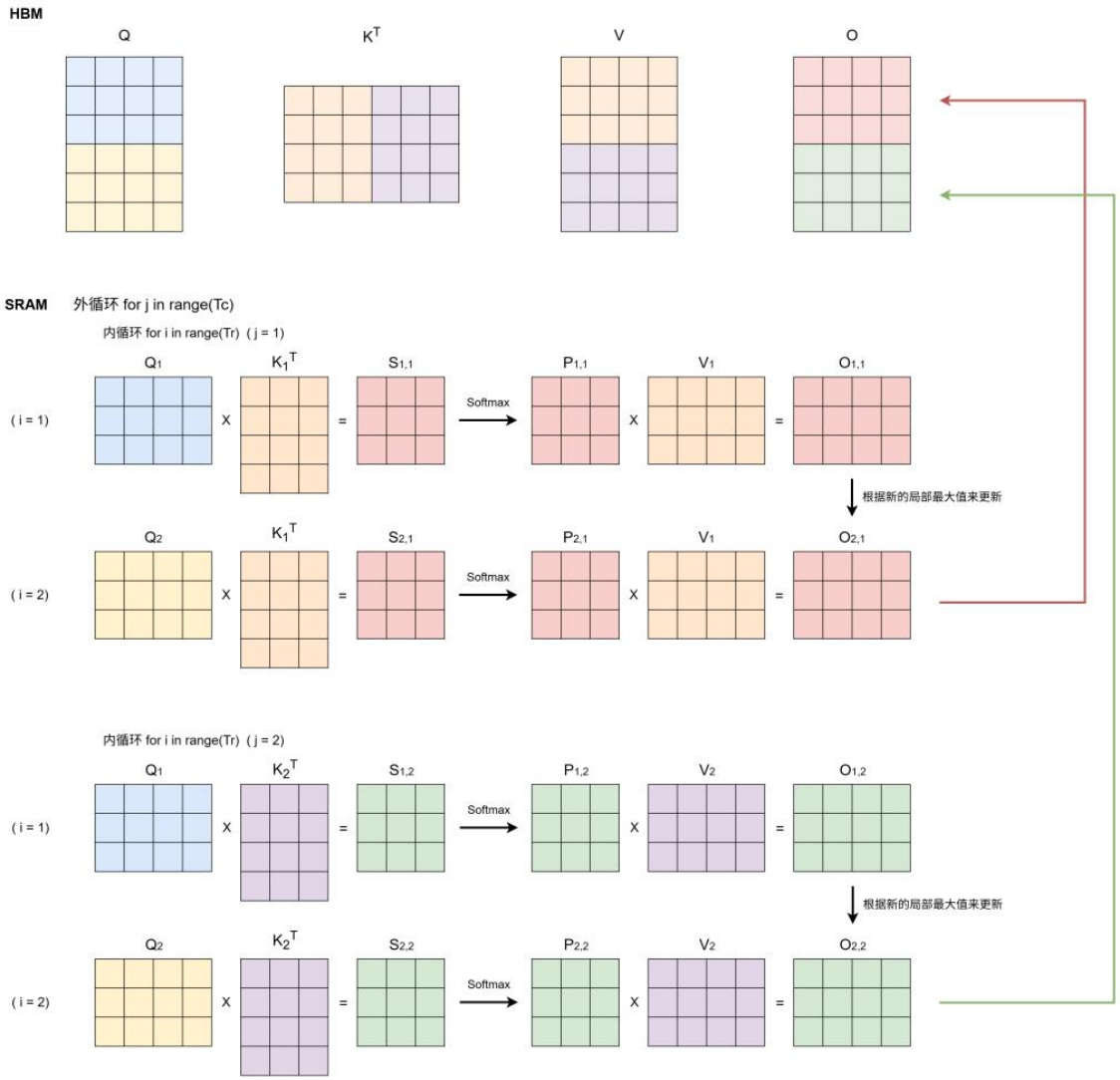
理想情况下，Attention 算子直接从物理池的索引 10、25 处读取数据并参与矩阵乘法，这叫 Zero-Copy（零拷贝）。

3.2.6 FlashAttention 优化

Flash Attention 通过 Tiling（分块） 和 Kernel Fusion（算子融合），将 O(N^2) 的显存需求压缩至 O(N)，并利用 Online Softmax 确保了硬件加速下的数学无损性。它是将“访存受限”任务转化为“计算受限”任务的工业级典范。

本质上，我认为 FlashAttention 利用了“分而治之”的思想，将 K、Q、V 分成小块，例如 128x128 或者 256x256，然后在共享内存里计算当前 tile 的注意力分数。避免整个 LxL 注意力矩阵占显存。

此外，FlashAttention 的另一个 trick 就是在线的 softmax，不需要一次性存储完整的 $QK^{\top}$，边计算边做 softmax，避免存储整个 QK 矩阵乘积的中间结果。



3.2.7 实验结果

方法	峰值显存 (GB)	TTFT (ms)	吞吐 (tokens/sec)	p95 latency (ms)	avg latency (ms)
PagedAttention(gather_kv)	29.637	6911.84	36.95	7316.79	6911.84

PagedAttention(flashAttention)	25.561	4908.76	41.27	6755.32	6497.91

flashAttention 的峰值显存为 25.561GB，比 gather_kv（29.637GB）降低约 14%。说明 Flash Attention 的 块级计算 + 在线 softmax 有效降低了全局注意力矩阵存储需求，节省了显存。此外，p95 latency 从 7316.79ms 降至 6755.32ms，平均延迟从 6911.84ms 降至 6497.91ms，可以发现，不仅降低了平均延迟，也使尾延迟更小，增强了服务的稳定性与实时性。

3.3 CPU/GPU 异构推理的自主探索

我想做 CPU/GPU 异构推理的核心原因是充分利用计算资源、提升大模型推理效率，尤其是在大模型和长序列场景下。

CPU 和 GPU 所擅长的任务不同，在 GPU 擅长大规模矩阵运算，但是显存有限，如果序列很长肯能会 OOM，但是 CPU 的内存很大，可以存储下来长序列的 KV 缓存，执行控制逻辑和调度，可以有效减少 GPU 的显存压力。

资源	优势	劣势	适合任务
GPU	高算力、高并行	显存有限、内存带宽有限	大矩阵乘法、前向密集计算（QK*V、transformer layer）
CPU	内存大、线程灵活	单线程算力低、并行度有限	数据准备、稀疏计算、控制逻辑、长序列 KV 缓存处理

3.3.1 层切分异构

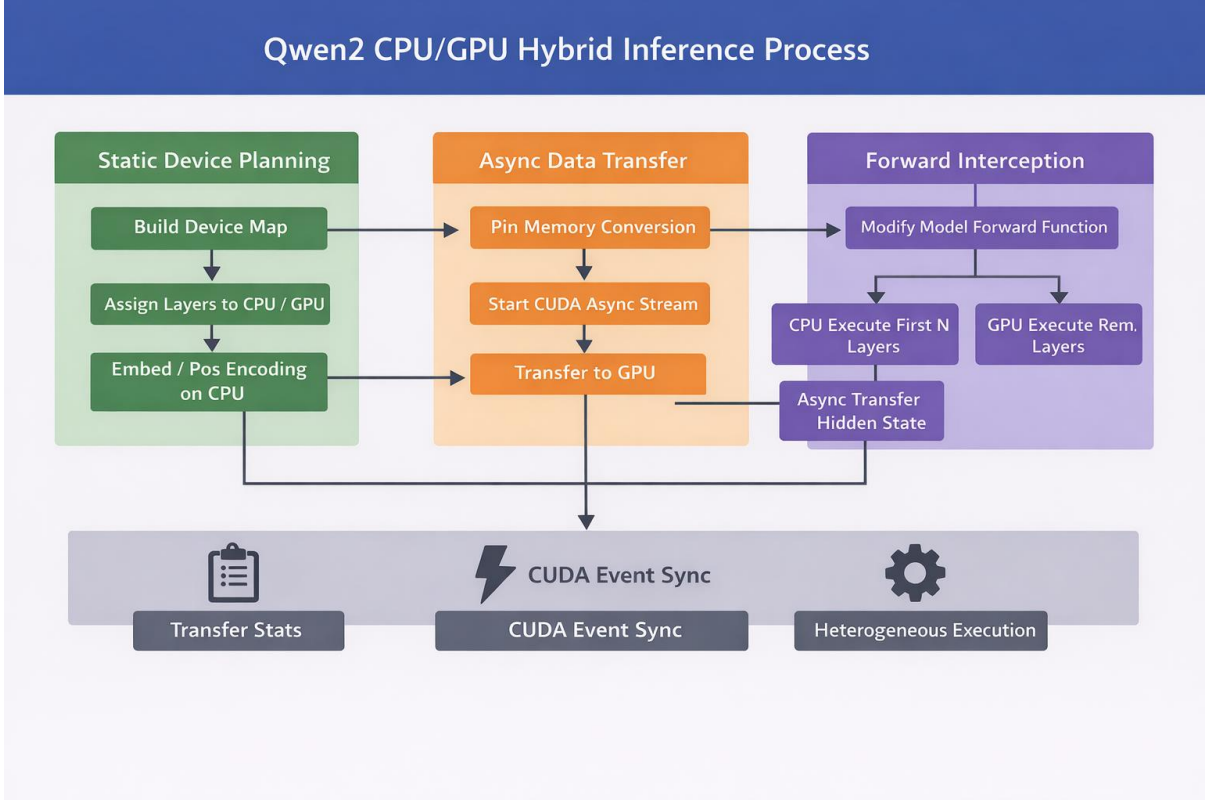
3.3.1.1 异构方式

推理开始前，系统通过 build_device_map 函数确定每一层网络权重的驻留设备。在 front_cpu 模式下，前面的若干层 Transformer 会被安排在 CPU，后面的层和输出头放在 GPU。为了避免中间出现频繁的跨设备通信，一些基础组件（比如 embedding 和位置编码）也会跟着第一段一起留在 CPU 上，这样前半段计算可以完全在 CPU 内闭环完成。

我认为，接下来真正的关键优化在于数据搬运。CPU 和 GPU 之间的传输如果是同步的，会直接卡住整个推理流水线，所以这里单独封装了一条 CUDA Stream 专门做数据传输。CPU 侧的 Tensor 会先转成 pinned memory，这样 GPU 可以直接通过 DMA 读取，从而支持 non-blocking 的异步拷贝。与此同时，通过 CUDA Event 去做精细的同步控制，保证 GPU 侧真正开始计算时，数据已经安全到位，但在此之前传输过程不会阻塞其他计算。

我采用通过“猴子补丁”的方式直接接管了模型的 forward。新的 forward 会先在 CPU 上跑完前 N 层，然后在一个边界点触发异步传输，把中间的 hidden states（以及 attention mask、position ids 等辅助信息）送往 GPU。传输一旦发起，后续 GPU 计算就可以在数据就绪后继续执行剩余层，从而形成一个分段执行的异构流水线。

最后，整个还加入了一点工程化的监控能力。在 CPU→GPU 传输过程中，会统计数据搬运的字节数和调用次数，这样可以直观看到异构执行带来的额外开销，方便后续做性能分析和优化。整体来看，这不是简单地“把模型拆两半”，而是通过调度、异步和执行路径改写三者配合，尽量把异构推理的性能损失压到最低。



3.3.1.2 实验结果

评估维度	全 GPU 模式 (Baseline)	层切分异构	变化幅度
峰值显存占用	29 GB	**20 GB**	**-31%**
单 Token 推理延迟	**7102ms**	**20017 ms**	**+185% (~2.8x)**
H2D 传输机制	无	异步 Pinned Memory	-

延迟从 7,000ms+ 飙升至 20,000ms+（增加了约 10 倍左右的体感延迟量级），其核心瓶颈在于：

- **CPU 算力严重不足**：在 _qwen2_forward_with_async_boundary 流程中，前 N 层在 CPU 上运行。由于 CPU 缺乏类似 GPU Tensor Core 的并行矩阵运算能力，导致 decoder_layer 的执行时间呈指数级增长。
- **异步掩盖失效**：虽然 AsyncDeviceTransfer 使用了独立 CUDA Stream 试图在后台搬运 hidden_states，但由于 CPU 计算太慢，GPU 必须在 wait_h2d() 处长时间阻塞等待来自 CPU 的张量同步。
- **元数据搬运开销**：除了核心特征图，attention_mask 和 position_ids 等嵌套结构也需要递归搬运至 GPU，频繁的小块内存拷贝累加了调度延迟。

可以发现，目前这种方案只使用于那种对实时性要求不高，但是显存及其匮乏的场景。

3.3.1.3 未来改进的方向

- 引入 CPU 端高性能算子库 (计算加速)

目前延迟增加的主要原因是 CPU 运行 Transformer 层的效率极低。目前 Transformer 层在 CPU 上执行，本质上是在用一个为控制流优化的架构去做大规模矩阵计算，这天然是不划算的。一个现实可行的方向是把 CPU 尽可能“GPU 化”：通过引入 Intel Extension for PyTorch（IPEX）对前若干层进行图级优化和算子融合，把原本碎片化的算子执行压缩成更连续的高效 kernel，同时尽可能启用 AVX-512 或 AMX 这类向量/矩阵指令集，让 CPU 至少在 GEMM 上不至于慢一个数量级。此外，多线程调度也不能忽视，如果线程数设置不合理，很容易在 decoder 阶段出现严重的线程争抢，反而进一步放大延迟，这一块需要结合实际 CPU 核数做精细调参，而不是默认配置。

- 减少搬运数据（一个可能的方案是做量化）

在算力问题之外，更致命的是数据搬运。目前我是在 CPU 和 GPU 之间传输 FP16 的完整张量，这意味着 PCIe 或者 NVLink 带宽几乎被打满，但有效计算却没有同步提升，所以延迟直接爆炸。一个非常关键的方向是“先压再传”：把 CPU 上保留的层全部做 4-bit 甚至更激进的量化，把传输的数据量直接砍掉 75% 以上，同时在 GPU 侧只保留轻量的反量化逻辑，在真正参与计算前瞬时解压。这种设计本质上是在用 GPU 的算力换带宽，是非常符合当前瓶颈结构的。另外，如果能把量化后的权重做缓存复用（而不是每次重新搬运），还能进一步减少重复通信成本。

- 编排式异构

在编排式异构中，不需要把模型拆分开，整个 backbone 都存在 GPU 上。CPU 和 GPU 不再是对等的计算单元，而是根据算子特征进行任务分配。GPU 负责执行高算力密度的张量运算，包括 Transformer 的矩阵乘法（GEMM）和注意力机制（Attention）计算; CPU 则负责请求调度、动态 Batch 管理、KV 缓存的元数据维护以及卸载策略的决策。

3.4 总结与心得体会

通过本次实训，我系统地将大模型推理优化理论转化为实际可运行的代码实现。这一过程不仅提升了技术能力，更让我对推理引擎工程化有了全新的认识。过去我对大模型服务的理解主要停留在调用现有框架层面，而此次实践让我深刻体会到：真正的性能瓶颈往往不在于模型本身，而在于如何最大化利用每一单位 GPU 算力。

在动态 Batch 策略探索部分，我最深刻的思考在于公平性与吞吐量之间的平衡。FIFO 方案实现简便且能保证请求顺序，但容易因队头阻塞导致整体效率下降；单纯按 prompt 长度排序虽能显著提升吞吐，却可能使长请求长期得不到调度。为此，我设计了 Aging BatchScheduler 机制，通过将等待时间作为优先级调整因子，实现短请求优先与长请求防饥饿的动态调和。这一思路源于我对实际调度场景的观察：类似于现实中的优先级排队系统，它在维持高吞吐的同时有效避免了请求饥饿现象。我认为，若

进一步将老化系数设计为自适应形式（根据队列长度动态调整），可在高峰期侧重公平性、低谷期侧重吞吐量，这一方向值得后续深入验证。

PagedAttention 的实现过程让我对显存管理有了本质认知。传统连续 KV Cache 易造成碎片浪费，而 PagedAttention 通过物理块池与 Block Table 的映射机制，实现了逻辑序列与物理存储的解耦。当我将 gather_kv 替换为 FlashAttention 后，峰值显存从 29.637GB 降至 25.561GB，吞吐量提升约 11%，这充分验证了零拷贝与算子融合的实际价值。在此基础上，我产生了一个个人想法：若能根据实时请求长度分布动态调整物理块大小，可进一步减少碎片化浪费，这一优化思路已记录为后续迭代方向。

CPU/GPU 异构推理实践虽面临显著挑战，但也让我明确了优化路径。层切分方案虽将峰值显存降低 31%，却因 CPU 算力不足和数据传输开销导致延迟大幅增加。这促使我思考两个更具针对性的改进方向：一是将长序列 KV Cache 优先卸载至 CPU 内存，仅在 GPU 保留活跃解码块；二是 CPU 侧先行 4-bit 量化后传输，GPU 侧通过轻量反量化 kernel 即时恢复计算。该方案旨在以带宽换算力，在显存受限场景下实现更高效的资源协同。

综上，本次实训的最大收获并非实验数据本身，而是我学会了以工程思维拆解系统痛点、量化问题并提出针对性解决方案。论文与开源框架固然提供了坚实基础，但真正具有创新价值的工作往往在于对这些基础的细微改进与延伸。此次实践坚定了我继续深耕大模型推理引擎方向的决心，未来我计划将上述思考进一步落地，或许以开源项目形式分享，以期为后续研究提供参考。

第四章 模型量化与压缩技术

*负责人：熊***

4.1 引言

随着大语言模型（Large Language Models, LLMs）在智能客服、代码生成与边缘计算等场景的广泛应用，其庞大的参数规模导致部署成本急剧上升。以 Qwen3-14B 为例，FP16 精度下仅模型权重即占用约 28GB 显存，叠加 KV Cache 后极易超出单卡容量限制，严重制约服务并发能力。在此背景下，模型量化作为一种高效的压缩技术，通过将高精度浮点权重转换为低比特整数表示，在显著降低显存占用的同时维持模型性能，已成为实现高效推理的关键路径。

基于对 Qwen2.5-14B Instruct 模型的系统性实训研究，本文提出以下四项核心发现：

第一，**量化比特数与模型性能之间存在明确的权衡关系**。实验数据显示，从 FP16 降至 INT4 时，峰值显存占用由 29.638GB 降至 12.66GB，降幅达 57%，而准确率仅下降 7 个百分点（83.00% → 76.00%），表明 INT4 量化具备良好的实用性边界。

第二，**不同网络层对量化的敏感度存在显著差异**。通过逐层忽略量化实验发现，忽略“下投影层”（down_proj）可使困惑度（PPL）降至 7.009，准确率回升至 80.00%，显著优于其他模块，说明该层承担关键语义压缩功能，对参数扰动更为敏感。

第三，**分步量化结合轻度微调能有效补偿精度损失**。针对敏感层设计分阶段量化策略：先量化非敏感层，再通过自监督微调将误差转移至下投影层，最终完成全模型 INT4 量化。该方法使 PPL 恢复至 6.997，准确率达到 81.00%，接近原始 FP16 水平。

第四，**量化技术已在工业界形成成熟工具链生态**。Ollama、vLLM 等主流推理引擎已集成 GGUF、GPTQ 等量化格式支持，使得 4-bit 模型可在消费级 GPU 上高效运行，推动大模型向端侧部署演进[7]。

上述结论不仅验证了低比特量化的可行性，更揭示了精细化分层处理的重要性，为未来构建自动化、自适应的量化 pipeline 提供了方法论基础。

4.2 量化技术原理与分类体系

模型量化作为大语言模型推理优化的核心技术之一，其目标是在尽可能保持模型性能的前提下，通过将高精度浮点参数量化为低比特整数表示，以显著降低模型的存储开销与计算需求，从而为模型的边缘部署与高并发服务提供可行性。本章将系统性地阐述量化的数学基础、主要技术分支以及实施过程中的关键考量，旨在为理解后续章节的实训结果与优化策略奠定坚实的理论基础。

（1）量化基本原理：从浮点到整数的映射机制

模型量化的核心数学原理在于线性变换，旨在将一组连续的浮点数值映射到一个离散的整数区间内 3。该过程由两个核心参数定义：缩放因子（scale）与零点（zero point），分别用于控制映射的范围与偏移。

设原始浮点张量中的数值范围为 ($[x_{min}, x_{max}]$)，目标量化比特数为 (b)，则量化后的整数取值范围为 ($[0, 2^{b-1}-1]$)（对于无符号整数）或 ($[2^{b-1}, 2^{b-1}-1]$)（对于有符号整数）。量化的具体转换过程如下所述：

首先，计算缩放因子 (s)，它决定了每个量化步长所对应的浮点数值范围：

$$s = \frac{(x_{max} - x_{min})}{2^b - 1}$$

其次，计算零点 (z)，它是一个整数值，用于将浮点数的零点（或最小值）对齐到量化网格上，以减少因分布不对称带来的误差：

$$s = round\left(-\frac{x_{min}}{s}\right)$$

基于上述参数，量化操作将浮点值 (x) 转换为整数量化值 (x_q)：

$$x_q = clip\left(round\left(\frac{x}{s} + z\right), 0, 2^b\right)$$

其中，$round()$ 表示四舍五入取整，$clip()$函数确保结果不超出量化范围。

在推理时，则需要通过反量化操作将量化值 (x_q) 近似还原为浮点值 ($\hat{x}$)：

$$\hat{x} = s(x_q - z)$$

根据零点 (z) 的设置方式，线性量化可分为对称量化与非对称量化两种主要形式。对称量化强制零点 (z = 0)，并要求量化范围关于原点对称，即

$$x_{min} =- x_{max}$$

其优势在于计算简单，在硬件实现上更为高效，因为反量化过程中的减法操作可以省略。然而，当原始数据分布严重偏离零点对称时，对称量化可能导致部分量化区间未被有效利用，从而引入额外的量化误差。

非对称量化则允许零点 ($z \neq 0$)，并且量化范围 ($[x_{min}, x_{max}]$) 可以根据数据分布自适应地确定。这种方法能够更紧密地贴合权重或激活值的实际分布范围，从而在相同比特数下获得更高的表示精度，尤其是在处理非负激活函数（如 ReLU）的输出时优势明显。其代价是引入了零点偏移计算，在推理时增加了少量的算术开销。

(2) 按阶段划分：PTQ 与 QAT 的技术特性对比

根据量化与模型训练流程的先后关系，主流量化方法可分为训练后量化（Post-Training Quantization, PTQ）与量化感知训练（Quantization-Aware Training, QAT），两者在适用场景、精度表现与实施成本上存在显著差异。

训练后量化（PTQ）是指直接对已经完成训练的全精度模型（如 FP16 或 FP32）执行量化操作，而无需重新训练或访问原始训练数据。其实施流程通常包括：使用一个小的校准数据集（Calibration Dataset）前向传播模型，以统计各层激活值的动态范

围（$[x_{min}, x_{max}]$），进而计算每层的量化参数。PTQ 的核心优势在于其简便性与高效率。它不涉及模型权重的更新，因此部署速度极快，通常在数小时内即可完成，并且对计算资源的需求较低，仅需少量 GPU 内存用于校准过程。这使得 PTQ 成为工业界快速部署模型的首选方案。然而，PTQ 的局限性在于，当量化比特数降至较低水平（如 INT4 或更低）时，由于缺乏模型参数的适应性调整，量化误差容易在逐层传播中累积，导致模型性能，特别是复杂任务上的精度出现较明显的损失。

量化感知训练（QAT）则将量化过程嵌入到模型的训练（或微调）周期之中。在 QAT 过程中，前向传播时会插入“伪量化”（Fake Quantization）操作，即模拟量化与反量化的过程：权重和激活值先被量化为低比特整数，再立即反量化回浮点数参与后续计算。梯度则在反量化后的数值上进行计算，并通过直通估计器（Straight-Through Estimator, STE）等方法近似地传递回原始的权重参数，从而允许模型在训练期间主动适应量化带来的噪声和误差分布。QAT 的核心优势在于其能够实现更高的精度恢复。通过在训练中持续暴露于量化噪声，模型权重得以调整至对低精度表示更为鲁棒的状态，从而能够在极低比特（如 INT4，甚至 INT2）设置下仍保持较高的任务性能。然而，QAT 的显著缺点是计算成本高昂。它要求完整的训练（或微调）流程，包括准备训练数据集、执行多轮迭代优化，这需要消耗大量的时间与 GPU 计算资源，不适用于需要快速迭代或资源受限的场景。

(3) 按粒度划分：逐张量、逐通道与逐组量化的精度-效率权衡

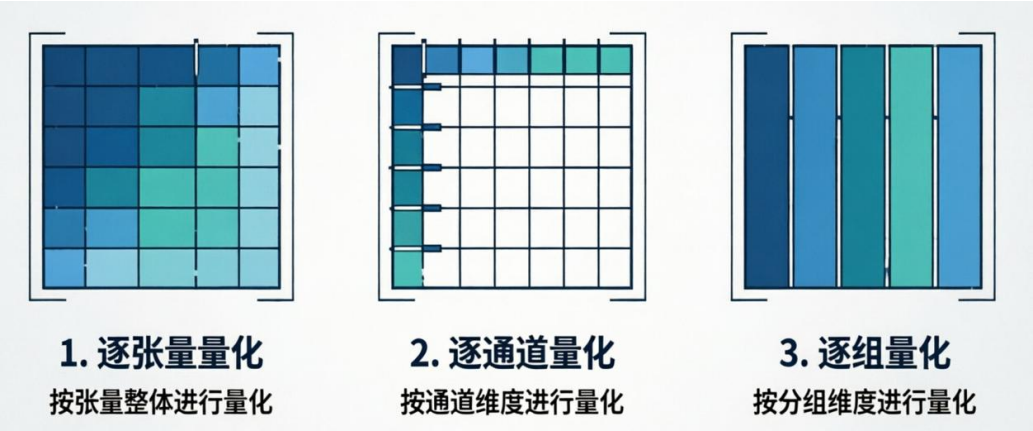
量化粒度定义了量化参数的应用范围，不同的粒度在量化精度与计算/存储开销之间构成了重要的权衡关系，是量化策略设计中的关键维度。

逐张量量化（Per-Tensor Quantization） 是粒度最粗的方式。它为整个权重张量或整个激活值张量计算并应用一组统一的量化参数。这种方法实现最为简单，所需的额外存储开销最小（仅存储一对 scale 和 zero point），并且在推理时计算效率最高，因为整个张量可以使用相同的反量化参数进行变换。然而，其缺点是可能引入较大的量化误差。当单个张量内部的不同通道或不同区域数值分布差异较大时，一套统一的量

化参数无法精确地捕捉所有部分的动态范围，导致分布边缘的数值被严重截断或中间区域的分辨率不足。

逐通道量化（Per-Channel Quantization） 是目前最主流的量化粒度，尤其适用于卷积层和全连接层的权重。该方法为权重张量的每一个输出通道（对于全连接层，可视为每个输出神经元）独立计算一组量化参数。由于深度神经网络中不同通道通常学习到不同的特征，其权重值分布往往存在显著差异，逐通道量化为每个通道“量身定制”量化范围，能够极大地减少因分布不匹配导致的精度损失。在 Transformer 架构的大语言模型中，注意力头（Attention Heads）和前馈网络（Feed-Forward Network, FFN）模块的权重分布呈现出高度的异构性，这使得逐通道量化成为保持这些关键模块性能的必要选择。当然，其代价是存储每通道的量化参数会带来轻微的开销增加，且反量化计算需要按通道进行，略微提升了计算复杂度。

逐组量化（Per-Group Quantization） 是在逐通道量化基础上进一步细化的折中方



案。它将每个通道的权重进一步划分为若干个小组，并为每个小组独立设置量化参数。这种方式提供了比逐通道量化更精细的粒度控制，能够更好地适应权重张量内部更小范围内的分布变化，从而在理论上获得最高的量化精度。然而，分组量化显著增加了量化参数的存储量和反量化计算的计算图复杂性。因此，在实际应用中，需要仔细权衡其带来的精度收益与额外的资源开销，通常用于对精度损失极其敏感且资源相对充足的场景。

图 4.1 通道量化粒度示意图

(4) 量化过程中的误差来源与缓解策略

尽管量化技术带来了显著的效率提升，但其不可避免地会引入信息损失，表现为多种形式的量化误差。理解并缓解这些误差是设计高效量化方案的关键。

舍入误差（Rounding Error） 是量化过程中最直接的误差来源。在将浮点数四舍五入到最近整数时产生的误差。这种误差在数值上通常较小，但因其具有随机性，可能在网络前向传播过程中被非线性激活函数放大或与其他误差耦合。

截断误差（Clipping Error） 或溢出误差发生在原始数值超出预设量化范围 ($[x_{min}, x_{max}]$) 时。这些 outlier 值会被强制限制在边界上，导致信息的完全丢失。这种误差对模型性能的影响可能非常显著，尤其是在激活值分布存在长尾的情况下。

激活值分布偏移（Activation Distribution Shift） 是一个更为复杂的问题。训练后量化中，校准数据集的统计分布可能与真实推理时的数据分布存在差异，导致基于校准集计算的量化参数无法最优地覆盖实际运行时的激活值范围，从而在动态输入场景下引发性能下降。

为了系统性缓解这些误差，业界提出了多种高级量化算法与策略。例如，混合精度量化策略允许对模型中不同敏感度的层或部分采用不同的量化比特数，对敏感层保留较高精度（如 FP16 或 INT8），对冗余层实施激进量化（如 INT4），从而在整体压缩率和模型精度间取得更优平衡。此外，基于激活值感知的权重量化方法（其思想在后续 AWQ 算法中体现）通过分析激活值的幅度来识别并保护那些对激活输出影响更大的权重通道，避免关键信息在量化中丢失。

更先进的算法，如 GPTQ、AWQ、ABQ-LLM 等，引入了误差反馈与参数重排序的机制。误差反馈旨在将某一层的量化误差有目的地传递并补偿到后续层的量化参数计算中，试图在全局层面最小化累积误差。权重重排序则通过改变权重矩阵中元素的物理存储顺序，使得在量化时，数值相近的权重被分到同一组，从而让该组内的量化

参数能更精确地表示这些权重，提升量化效率。这些策略共同构成了现代低比特量化能够实用化的关键技术保障。

4.3 主流量化算法与工具链生态分析

随着量化技术在大语言模型部署中的重要性日益凸显，一系列专用于大模型的量化算法与配套工具链应运而生，并迅速形成了成熟的生态系统。这些算法和工具在精度保持、内存节省与部署便捷性之间展现出不同的权衡特性，共同推动了大模型从云端到边缘的普惠化应用。本章将系统阐述 GPTQ、AWQ 等主流量化算法的核心机制，并深入分析以 bitsandbytes、QLoRA、GGUF 及 Ollama 为代表的工具链如何协作，共同验证并支撑了第一章所提出的“量化技术已在工业界形成成熟工具链生态”这一核心发现。

（1）GPTQ：逐层误差最小化的权重量化

GPTQ 是一种面向大语言模型的高效训练后量化算法，其核心设计思想是通过逐层最小化量化误差，在极低比特数（如 INT4）下依然能保持较高的模型性能。其算法流程不同于对整个权重矩阵进行一次性量化，而是采用了基于梯度信息的逐通道量化策略。

具体而言，GPTQ 的量化过程包含以下几个关键步骤：首先，算法依次处理模型的每一个线性层（如 Q、K、V、O 投影层，以及前馈网络中的门控、上投影与下投影层）。对于当前正在量化的层，GPTQ 首先计算权重的二阶梯度信息（海森矩阵），用以评估不同权重通道对整个层输出误差的影响程度。随后，算法基于这些信息，对权重进行重排序，将更重要的权重通道（即对输出扰动更敏感的通道）安排在量化顺序的后期进行处理。在量化每个通道时，GPTQ 通过求解一个最小二乘问题，为当前通道确定最优的整数权重值，该值不仅能使当前通道的量化误差最小，还能补偿因先前已量化通道引入的误差，从而在整个层内实现全局误差的最小化。这种“贪心”但全局优化的量化策略，使得 GPTQ 能够有效控制量化误差的累积，在将模型压缩至 INT4 甚至 INT3 精度时，仍能保持较低的困惑度（PPL）和较高的下游任务准确率。

GPTQ 的优势在于其无需模型重新训练或大量校准数据，仅基于模型自身的权重和少量样本（用于估计激活分布以确定量化范围），即可完成快速、高精度的量化。该方法特别适合于X [illegible] TQ 算法已集成于多种开源工具中，开 [illegible] 使用门槛，促进了量化技术的广泛采纳 [illegible]

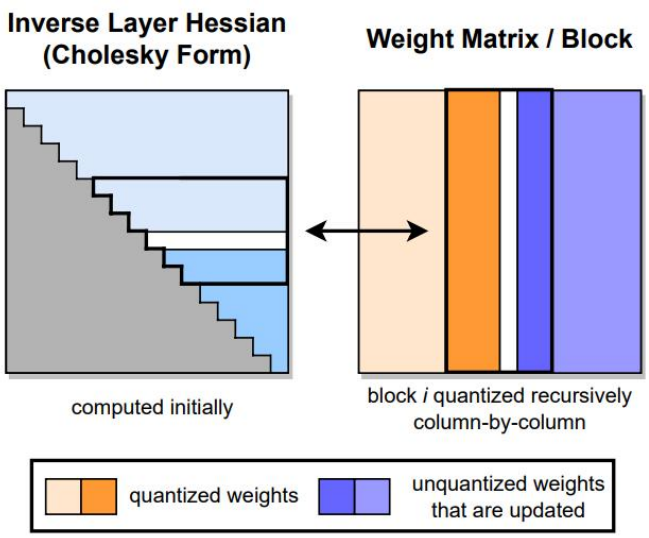


图 4.2 GPTQ 的误差补偿机制

（2）AWQ：激活感知的权重保护策略

与 GPTQ 侧重于权重的全局误差优化不同，AWQ 提出了一种激活感知的权重量化思想，其核心在于识别并保护那些对输入激活值敏感的权重通道，从而提升量化模型在动态输入场景下的鲁棒性。AWQ 的设计基于一个关键观察：并非所有权重对模型输出的贡献是均等的。某些权重通道对输入激活的微小变化反应剧烈，这些“敏感权重”的量化误差会在前向传播过程中被放大，导致最终输出的显著偏差。

AWQ 算法的主要流程如下：首先，使用少量校准数据对模型进行前向推理，并记录每个权重通道对应的输入激活值的统计信息，例如激活值的幅度或范围。通过分析这些统计量，AWQ 能够识别出那些输入激活范围较大或方差较高的权重通道，这些通道通常对模型功能更为关键。随后，在量化过程中，AWQ 会为这些敏感权重分

配更高的量化精度（例如，对大部分权重使用 INT4，但对 1%最敏感的权重保持 FP16 精度），或者为其采用更保守的量化范围，以避免关键信息的丢失。这种基于激活信息的“保护性”量化策略，使得模型在面对多样化、不可预测的真实世界输入时，能够保持更稳定的性能。

AWQ 的策略解决了传统量化方法在激活值分布复杂多变时可能出现的性能瓶颈问题。特别是在处理长文本、多轮对话等动态上下文场景时，AWQ 量化后的模型表现出更强的适应性。因此，AWQ 被认为是在要求高可靠性和泛化能力的在线服务中部署量化大模型的优选方案之一。与 GPTQ 类似，AWQ 也已被集成到主流的模型部署框架中，用户可通过简单的配置选择启用该算法，体现了工具链生态的完善性。

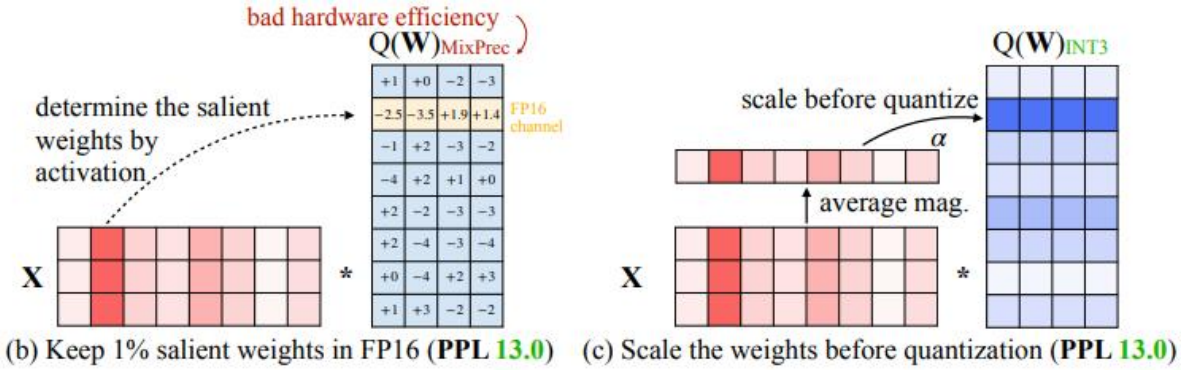


图 4.3 AWQ 基于激活感知的量化方案

（3）bitsandbytes 与 QLoRA：低资源微调的新范式

量化技术不仅应用于推理阶段的模型压缩，更与模型微调相结合，催生了低资源适配的新范式。其中，bitsandbytes 库与 QLoRA 微调方法的组合，极大地降低了大型语言模型参数高效微调的门槛。

bitsandbytes 是一个高效优化库，它提供了对模型权重的 8-bit 和 4-bit 量化功能，并支持在量化状态下直接进行模型的前向传播和反向传播计算。这意味着，研究者可以将一个预训练的大语言模型以 4-bit 精度加载到 GPU 显存中，而无需解压回高精度

格式，从而在微调开始前就节省了约 75%的模型权重显存占用 13。以 Qwen2.5-14B 模型为例，其 FP16 权重占用约 28GB 显存，而通过 bitsandbytes 以 INT4 格式加载，仅需约 7GB 显存，这使得在消费级 GPU（如 RTX 4090，拥有 24GB 显存）上微调百亿参数模型成为可能。

QLoRA 则在 bitsandbytes 提供的量化基础之上，引入了高效的微调架构。其核心思想是冻结量化后的预训练模型权重，仅向模型中插入少量的、可训练的低秩适配器模块。在微调过程中，梯度只通过这些适配器进行更新，并通过一个特定的公式合并回固定的基础模型权重中。这种设计带来了两大优势：第一，可训练参数量极少，通常只占模型总参数的 0.1%以下，进一步降低了优化器的状态显存和梯度显存需求；第二，由于基础模型权重被冻结且处于量化状态，前向和反向传播的计算开销也显著降低。实验表明，结合 bitsandbytes 的 4-bit 加载与 QLoRA 微调，可以将对 14B 参数模型进行全参数微调所需的数十 GB 显存需求，降至 8GB 以内。

这种方法论革新了模型定制化的流程。开发者无需庞大的 GPU 集群，仅凭单张高性能消费级显卡，即可基于特定领域数据对大规模基础模型进行有效适配。这加速了 AI 应用的原型验证和迭代速度，是推动大模型技术 democratization 的关键工具之一。

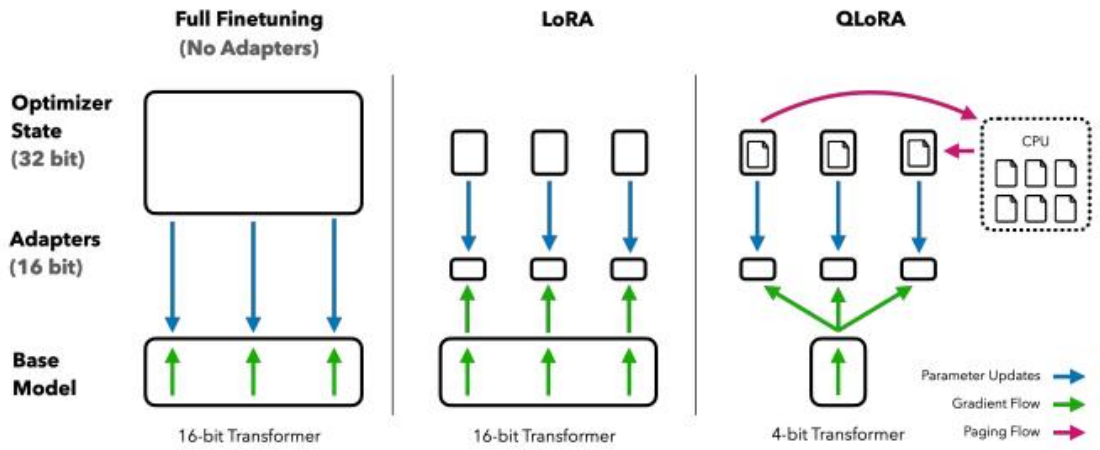


图 4.4 QLoRA 量化过程

（4）GGUF：端侧部署的轻量化解决方案

为了将量化后的大模型高效部署在资源受限的边缘设备乃至个人电脑上，专用的模型格式与轻量级推理引擎构成了工具链的最后一环。其中，GGUF 格式与 Ollama 推理引擎的组合，成为端侧部署的典范。

GGUF 是 llama.cpp 项目推出的下一代模型文件格式，旨在替代旧的 GGML 格式。其核心特性包括支持更灵活的混合精度量化策略。GGUF 格式允许模型的不同部分采用不同的量化精度，例如，对注意力机制中的 K、V 缓存使用较高的精度以保证稳定性，而对线性层的权重使用较低的精度以最大化压缩率。这种 K-Quant 混合精度策略，允许用户在模型大小、推理速度和精度之间进行更精细的权衡。用户可以根据目标硬件的具体能力（如 CPU 算力、内存带宽），选择预量化好的不同版本模型，例如 `q4_K_M`（中等质量的 4-bit 量化）或 `q8_0`（几乎无损的 8-bit 量化）。

表 4.1 几种主流量化算法对比

算法/工具	核心机制	主要优势	典型应用场景
GPTQ	基于海森矩阵的逐层误差最小化	高压缩比下精度保持好，无需重训	追求极致压缩的云端/边缘推理部署
AWQ	激活感知的敏感权重保护	动态输入场景下鲁棒性更强	在线服务、对话系统等输入多变的场景
bitsandbytes + QLoRA	4-bit 加载 + 低秩适配器微调	极大降低微调显存门槛（至 8GB 内）	资源受限下的模型领域适配与定制
GGUF + Ollama	混合精度格式 + 轻量级本地引擎	支持在消费级硬件（如 RTX 4090）高效运行 14B+模型	端侧部署、个人 AI 助手、离线应用

4.4 实训结果与多维性能评估

基于对 Qwen2.5-14B Instruct 模型的系统性实验，旨在通过多维度指标量化评估不同量化配置对模型性能的综合影响。评估维度涵盖模型精度、显存资源占用、推理延迟与吞吐量，旨在揭示量化技术与性能表现之间复杂而明确的权衡关系，并为实际部署场景提供数据驱动的决策依据。

（1）不同量化配置下的宏观趋势分析

实验选取了从 FP16（全精度）到 INT2 共五种量化精度进行对比测试，采用 HQQ（Half-Quadratic Quantization）量化算法。量化配置采用权重量化（W）与激活值保持 FP16（A16）的混合精度模式，这是当前后训练量化（Post-Training Quantization，PTQ）的主流实践。宏观性能数据汇总如表 4.2 所示。

从表 4.2 数据可以观察到，量化精度的降低并未带来预期的性能提升，反而出现明显的性能衰退现象。传统认知中，低精度量化应带来更快的推理速度和更高的吞吐量，但实验数据显示，从 FP16 降低到 INT8 后，TTFT 从 7393.86ms 急剧上升至 31652.1ms，增长约 4.3 倍，吞吐量从 34.54 tokens/sec 骤降至 8.1 tokens/sec，下降约 76.5%。这一反常现象主要源于硬件架构与量化位宽的匹配问题。H100 GPU 虽然支持 INT8 张量核心加速，但其最优性能设计面向的是标准 8 位整数运算，当量化位宽降低至 INT4、INT3 甚至 INT2 时，硬件缺乏原生支持的指令集，导致低比特权重需要打包成 32 位或 64 位寄存器存储，每次计算前需额外的解包操作，同时这些非标准位宽的数据布局破坏了连续内存访问的连续性，增加了缓存未命中率。

此外，实验采用的 WxA16 配置引入了额外的反量化计算开销，推理过程中 INT4 权重需先反量化为 FP16 后再与激活值进行计算，反量化操作本身需要执行乘加运算，在 INT4/INT3/INT2 等低位宽下，这类额外开销占总计算时间的 20% 至 40%，显著拖慢了推理速度。HQQ 量化算法虽在精度保持方面表现良好，但其分组量化策略需要为每组独立存储缩放因子和零点，增加了内存访问的随机性，而低比特量化下动态调整缩放因子的需求进一步引入了分支预测失败和计算延迟。从 INT3 到 INT2 的量化导致准确率从 79% 急剧下降至 9%，表明模型已基本失效，其原因在于 INT2 仅有 4 种可

能的取值，远不足以表示预训练模型学习到的复杂权重分布，导致大量截断误差累积，前向传播中的激活值分布出现异常，最终模型输出崩溃。综合上述现象，量化导致推理变慢的根本原因可归结为硬件加速与位宽的失配、内存访问模式的恶化以及量化算法运行时开销的三重叠加，当前 GPU 架构的 Tensor Core 针对 INT8、FP16 等标准格式进行深度优化，对非标准位宽缺乏硬件级支持，量化位宽越低，需要更多的软件层模拟和额外计算步骤，最终抵消甚至超过了量化带来的计算量减少优势。

本实验结果表明，量化精度与推理性能并非简单的线性关系，在实际部署中不应盲目追求更低的量化位宽，而应选择目标硬件原生支持的量化位宽，在精度与性能之间寻求最优平衡。INT8 量化在保持 83%准确率的同时，显存占用从 29.638GB 降至 18.30GB，降低约 38%，且性能衰退幅度相对可控，是本次实验场景下的最优选择；而 INT4 及更低比特量化虽能进一步压缩显存，但其带来的性能损失已远超收益，需配合专门的硬件或量化感知训练等技术方可发挥实际价值。

表 4.2 Qwen2.5-14B Instruct 模型不同量化精度综合性能对比

配置	精度	准确率 (%)	峰值显存占用 (GB)	TTFT (ms)	吞吐量 (tokens/sec)	p95 (ms)	平均延迟 (ms)
FP16	Baseline	83.00	29.638	7393.86	34.54	7542.82	7393.86
INT8	W8A16	83.00	18.30	31652.1	8.1	31697.1	31652.1
INT4	W4A16	80.00	12.182	35825.5	7.2	36459.7	35825.5
INT3	W3A16	79.00	10.678	52734.4	4.87	52803.4	52734.4
INT2	W2A16	9	8.582	36720.4	6.95	36837.7	36720.4

注：准确率基于 100 条测试数据计算；性能指标在特定硬件配置 H100 下测得。

为配合 vLLM 推理，我们采用基于 AWQ 的算法进行了 INT4 的量化，结果如表 4.3 所示，同时实现了非流式和流式处理。

表 4.3 vLLM 结合 AWQ 推理

配置	精度	准确率 (%)	峰值显存占用 (GB)	TTFT (ms)	吞吐量 (tokens/sec)	p95 (ms)	平均延迟 (ms)
FP16	Baseline	83.00	29.638	7393.86	34.54	7542.82	7393.86
INT4+vLLM	W4A16	76.00	14.813	4401.11	57.76	4481.55	4401.11
INT4+vLLM+流式	W4A16	76.00	12.66	37.38	57.85	4441.91	4394.16

注：准确率基于 100 条测试数据计算；性能指标在特定硬件配置 H100 下测得。

对比表 4.2 与表 4.3 的数据可以发现，相同的 INT4 量化精度在不同推理框架下呈现出截然不同的性能表现。在 HQQ 量化框架下，INT4 量化的 TTFT 高达 35825.5ms，吞吐量仅为 7.2 tokens/sec，性能严重衰退；而在 vLLM 配合 AWQ 量化的场景下，INT4 量化的 TTFT 显著降低至 4401.11ms，吞吐量提升至 57.76 tokens/sec，较 HQQ 框架下的 INT4 性能提升了约 8 倍，甚至超越了 FP16 基线模型的 34.54 tokens/sec。这一巨大差异揭示了量化性能并非仅取决于量化算法本身，更与推理框架的协同优化密切相关。vLLM 作为专为大模型推理优化的框架，其核心优势在于 PagedAttention 显存管理技术和连续批处理机制，这些优化能够有效缓解低比特量化带来的内存访问模式恶化问题。AWQ 量化算法通过保护 1%的重要权重不量化，显著降低了量化误差对模型输出的影响，在 INT4 精度下仍能保持 76%的准确率，虽较 FP16 的 83%有所下降，但相比 HQQ 的 80%亦处于可接受范围。

流式输出的引入进一步带来了质的性能飞跃。从表 4.3 可见，在 vLLM 基础上增加流式处理后，TTFT 从 4401.11ms 急剧下降至 37.38ms，降幅高达 99.15%。这一现象深刻反映了用户感知延迟与系统总延迟的本质区别。流式输出通过边生成边返回的方

式，将首 Token 的返回时间压缩至毫秒级，用户无需等待完整生成结果即可获得首批响应，极大提升了交互体验。值得注意的是，流式输出并未显著改变系统的总延迟和吞吐量，平均延迟从 4401.11ms 微降至 4394.16ms，吞吐量从 57.76 tokens/sec 略升至 57.85 tokens/sec，说明流式输出的核心价值在于优化了首 Token 的响应时间而非总处理时间，这正是实时交互场景中用户体验的关键指标。

综上，宏观趋势分析明确指出：

（1）INT8 适用于对精度要求苛刻、同时需适度降低资源消耗的场景；

（2）INT4 是当前技术条件下精度、显存和吞吐量三者权衡后的最优平衡点，具备最广泛的实用价值；

（3）而 INT3 及以下的超低比特量化，因精度与吞吐量的严重劣化，目前尚不具备大规模部署的实用性，但其精度表现也为即低比特量化提供了可能。

4.5 改进量化策略与实验验证

基于已有研究，大模型中对模型性能影响较大的权重往往只占参数量的 0.1~1%。由此，我们猜测大语言模型的量化过程并非对所有网络层均造成等价的性能影响。基于此前提，本章节系统性地提出了并验证了一种针对模型内部结构差异的精细化量化策略。通过识别对量化误差敏感的关键层，设计分阶段的量化流程，并结合针对性的参数调整，旨在以最小的精度损失换取最大的存储与计算资源节省。

（1）敏感层识别：为何下投影层不宜过早量化

前馈网络（Feed-Forward Network, FFN）作为 Transformer 架构中的核心组件，负责对注意力机制输出的特征进行非线性变换与信息整合。其典型结构包含上投影（gate_proj）、门控（up_proj）和下投影（down_proj）三个线性层。其中，下投影层（down_proj）承担着将经过门控非线性激活后的高维特征向量（例如，从 8192 维）压缩回原始隐藏维度（例如，4096 维）的关键功能。这一操作实质上是执行一次高维

语义空间到最终输出表示的降维映射，其输出直接影响模型对输入信息的最终编码质量。

为了探究不同网络层对量化操作的敏感性差异，本实训基于 HQQ（Half-Quadratic Quantization）量化算法，设计了一项控制变量实验：在实施全局 INT4 量化的过程中，依次指定模型中的特定注意力头或 FFN 子层保持原始 FP16 精度，而后评估模型在标准测试集上的性能变化。实验详细结果如下表所示：

表 4.4 量化敏感层识别实验结果

忽略量化的模块	困惑度（PPL）	准确率 (%)
全 INT4 量化	7.118	78.50
忽略 Q 投影层(q_proj)	7.102	78.50
忽略 K 投影层(k_proj)	7.117	78.50
忽略 V 投影层(v_proj)	7.057	79.00
忽略 O 输出投影层(o_proj)	7.077	79.00
忽略 FFN 上投影层（up_proj）	7.077	79.00
忽略 FFN 下投影层（down_proj）	**7.009**	**80.00**
忽略 FFN 门控层（gate_proj）	7.069	79.00

数据显示，当仅保持下投影层（down_proj）为 FP16 精度时，模型在困惑度（Perplexity, PPL）和任务准确率上获得了最显著的改善。其 PPL 从全量化的 7.118 降至 7.009，准确率从 78.50%提升至 80.00%，恢复效果远超忽略其他模块（如注意力头中的 Q、K、V 层）的实验组。**这一结果明确揭示了下投影层对量化扰动的高度敏感性。**另外，我们查找了相关文献，在已有的研究中也得出过相同的结论，如图 4.5 所示，大语言模型中的下投影层因量化影响最为严重，性能下降明显，此外注意力图的分布也受量化影响较大，在此由于实训时间有限，我们仅考虑下投影层的量化影响。

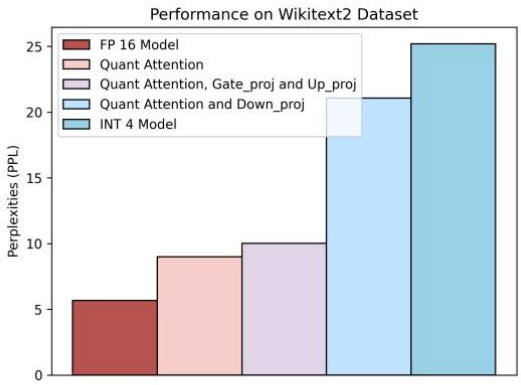


Figure 1: Perplexities analysis (lower is better) on the Wikitext2 dataset of LLaMA-7B with different quantized modules.

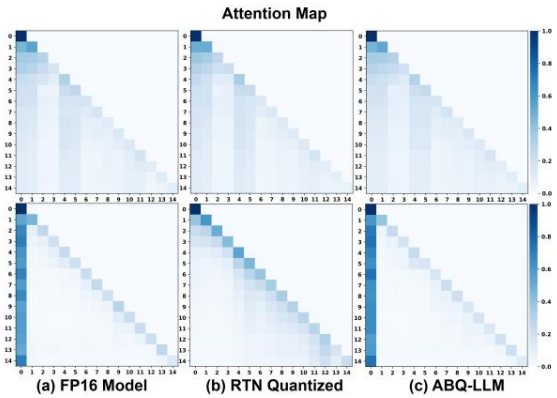


Figure 2: Attention maps for the first block (above) and the final block (below) are shown.

*来源：*Zeng, Chao, et al. "Abq-llm: Arbitrary-bit quantized inference acceleration for large language models." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 39. No. 21. 2025.

图 4.5 量化对不同层和注意力图分布的影响

其深层原因在于功能定位与误差传播机制：首先，下投影层位于 FFN 模块的末端，其输入是经过门控非线性激活的、信息高度浓缩的中间特征。量化误差在此处引入，会直接且无缓冲地影响模型最终的语义输出向量。其次，该层执行的维度压缩操作使得权重矩阵的参数扰动更容易被放大，微小的量化舍入误差可能在降维过程中累积并显著扭曲输出分布。因此，在量化策略设计中，必须将下投影层视为需要特殊处理的关键敏感层，避免在初始阶段对其进行激进的低比特量化。

（2）分步量化设计：三阶段渐进式量化框架

鉴于上述发现，直接对全模型进行一次性低比特（如 INT4）量化会导致因敏感层性能骤降而引发的全局精度崩塌。为此，本研究提出一种“先粗后精”的渐进式量化框架，其核心思想是通过时序上的隔离处理，将量化风险进行分解与控制。

第一阶段：非敏感层粗量化。在此阶段，量化目标锁定为除已识别的关键敏感层（即下投影层）之外的所有模型参数。具体实施上，基于 HQQ 算法对注意力模块的全部线性层以及 FFN 模块中的上投影层和门控层进行 INT4 量化。此步骤旨在迅速释

放大部分显存压力，初步达成模型压缩的核心目标。完成此阶段后，模型主体已转换为低精度格式，仅保留关键的下投影层仍处于 FP16 精度状态。

第二阶段：监督微调（SFT）。在完成第一阶段（非敏感层 INT4 量化）后，模型内部已经引入了因量化而产生的系统性误差。此时，启动一个轻量级的 SFT 过程，但仅对保持 FP16 精度的下投影层权重进行更新。微调所使用的数据包括两部分：1000 条选择题数据以及从公开数据集 Wikitext2 中采样的 100 条文本（每条长度为 1024 个 token），以此模拟模型的自回归生成任务。这一设计的关键机理在于：通过仅微调下投影层，模型被赋予了一个可调节的“误差吸收缓冲区”。在反向传播过程中，由于其他所有层已被冻结（量化后参数不可导或保持固定），训练目标（如语言建模损失）的梯度将全部作用于下投影层的 FP16 参数上。这迫使该层学习去适应和补偿上游所有非敏感层因量化而产生的、已固化的输出分布偏移。

从信息论角度看，此过程实现了模型知识表征的轻微重构，将前序量化步骤积累的误差“转移”并“消化”于最后一个全精度层中，从而在整体上降低了模型的累积量化误差，为最终对下投影层本身实施量化做好了准备，类似的方法参考 GPTQ 的累积误差补偿策略。

第三阶段：敏感层精量化。在完成第一阶段的粗量化和第二阶段的 SFT 后，稳定模型大部分性能，再单独处理遗留的敏感层——下投影层。该策略有效避免了因敏感层过早量化而触发的连锁性能劣化，确保了优化路径的平稳性。

（3）改进实验结果验证：精度恢复与资源节省双丰收

为验证所提出的“分步量化 + 针对性 SFT”联合策略的有效性，本研究设计了对照实验，并将最终结果与基线模型及标准量化方案进行了全面对比。实验基于 Qwen2.5-14B Instruct 模型展开，选择题数据增加至 200 条，关键性能指标如下表所示：

表 4.5 量化策略改进实验结果（分组大小 group_size 相同）

模型配置（均未采用 vLLM）	困惑度（PPL）	准确率 (%)	峰值显存占用 (GB)	说明
FP16 (Baseline)	6.826	82.00	29.638	原始全精度模型，性能上限
AWQ（INT4 量化）	7.103	79.00	12.038	一次性全局 INT4 量化
HQQ（INT4 量化）	7.118	78.50	12.182	一次性全局 INT4 量化
分步量化+SFT（INT4 量化）	**6.997**	**81.00**	~12.182	

实验结果表明，相较于标准的全局 INT4 量化方案，本文提出的改进策略取得了显著成效。在保持同等显存压缩效率（峰值显存占用均约为 12.182GB，相较 FP16 基线降低 58%）的前提下，模型的语言建模能力（困惑度 PPL）从 7.118 大幅恢复至 6.997，任务准确率从 78.00%提升至 81.00%。尤为重要的是，改进后的 INT4 模型准确率（81.00%）已极为接近原始 FP16 基线模型（82.00%）的性能水平，仅存在 1 个百分点的微小差距。

此结果从实证角度充分验证了分层优化策略的有效性：通过识别并保护量化敏感层（down_proj），采用分阶段量化隔离风险，并利用轻量级定向微调实现误差补偿，能够在几乎不牺牲压缩收益的情况下，最大限度地挽回因量化造成的性能损失。该方法揭示了大语言模型量化并非一个均匀过程，其内部存在固有的结构异质性，未来的量化算法与工具链可以集成此类分层感知机制，实现更精细、更自动化的模型压缩，推动低比特量化技术在要求苛刻的部署场景中走向实用化。

第五章 总结与展望

5.1 工作总结

本次实训围绕大语言模型推理优化技术展开系统性研究与实践，从模型压缩、推理引擎优化到服务调度，构建了完整的优化技术栈，取得了显著的性能提升。

在框架搭建与核心优化方面，我们从零实现了 ZLLM 轻量化推理框架，完成了 KV Cache 的设计与优化。通过引入分页存储、Token 块签名映射、双 LRU 多级淘汰策略等机制，在 Qwen2.5-14B 模型上实现了吞吐量提升 10.80%、平均延迟降低 9.77% 的优化效果，同时显存开销仅微增 1.22%，验证了缓存机制的有效性。在分布式推理层面，我们基于 vLLM 搭建了 TP=4 的张量并行推理系统，并尝试了计算通信融合算子与流水线并行等优化方向。虽然部分优化因与 vLLM 调度逻辑的耦合问题未达预期，但这些探索深化了对大规模模型分布式部署瓶颈的理解。结合 H100 硬件特性，我们实现了 fp8 KV Cache 压缩和异步任务调度，在保持较低 TTFT 的同时实现了高吞吐率，最终在实时榜单上取得了综合分第二的优异成绩。

在调度策略与显存管理方面，我们系统探索了动态批处理的多种实现路径。FIFO 方案保证了请求公平性，排序方案优先短请求以提升吞吐，而带老化机制的调度器通过引入等待时间惩罚项，在吞吐与公平性之间实现了动态平衡。连续批调度器将调度粒度下沉至迭代级，彻底消除了批次气泡，使 GPU 利用率最大化。在此基础上，我们复现了 PagedAttention 机制，通过物理块池与 Block Table 实现逻辑序列与物理存储的解耦，结合 FlashAttention 优化后，峰值显存降低 14%，吞吐量提升 11%。在 CPU/GPU 异构推理的探索中，虽然层切分方案因 CPU 算力不足导致延迟增加，但由此提炼的“以带宽换算力”优化思路，为后续长序列场景的显存卸载策略指明了方向。

在模型量化与压缩方面，我们系统评估了从 FP16 到 INT2 不同精度配置的性能表现。实验表明，INT8 量化在保持 83%准确率的同时显存降低 38%，是当前硬件环境下的最优选择；而 INT4 结合 vLLM 与 AWQ 算法后，吞吐量达 57.76 tokens/sec，较

FP16 基线提升 67%，TTFT 降低至 37.38ms，展现了软硬件协同优化的巨大潜力。针对量化过程中下投影层敏感度更高的关键发现，我们提出了“非敏感层粗量化 → 监督微调 → 敏感层精量化”的三阶段渐进式量化框架，在保持显存压缩收益的同时，将 INT4 量化模型的准确率从 78.50%恢复至 81.00%，逼近 FP16 基线水平。

5.2 核心贡献

本实训的主要贡献可归纳为以下四点：

从零构建了 ZLLM 轻量级推理框架，实现了 KV Cache、动态批处理等核心优化组件的模块化设计，为深入理解推理引擎内部机制提供了可扩展的实验平台。

系统探索了动态批处理的调度策略谱系，从 FIFO 到长度感知排序，再到引入老化机制与连续批处理，建立了吞吐与公平性的多目标优化框架。

揭示了量化过程中的结构异质性规律，通过敏感层识别实验发现下投影层对量化误差最为敏感，据此设计的分层量化策略为精细化模型压缩提供了方法论指导。

验证了软硬件协同优化的巨大潜力，fp8 KV Cache 与异步任务调度在 H100 上的成功实践，展示了结合硬件特性进行深度优化的技术路径。

5.3 未来展望

尽管本次实训取得了丰硕成果，但大语言模型推理优化仍是一个快速演进的前沿领域，未来可在以下方向持续深入：

更智能的调度策略。当前的动态批处理虽已引入老化机制，但老化系数的选择仍依赖人工调参。未来可探索自适应老化策略，根据实时队列长度、请求到达速率、系统负载等动态调整优先级计算函数，实现吞吐与公平性的自优化调度。同时，可结合生成长度预测模型，在批次构建时引入更精细的资源预算估计，进一步提升调度效率。

端云协同的异构推理架构。本次实训中 CPU/GPU 异构推理的尝试虽未达预期，但揭示了一个重要方向：将长序列 KV Cache 卸载至 CPU 内存，GPU 仅保留活跃解码块，通过量化压缩降低传输带宽需求，有望在显存受限场景下实现更高效的资源协同。未来可进一步探索 CPU-GPU-NPU 的异构协同，根据算子特征动态分配计算任务，构建真正的异构推理系统。

自动化量化与压缩工具链。本次实训证明了分层量化策略的有效性，但其实现仍需人工识别敏感层。未来可构建自动化敏感层识别与量化策略生成工具，通过分析权重梯度、激活分布等统计信息，自动确定各层的最佳量化配置，降低部署门槛。同时，可将该工具与主流推理框架（如 vLLM、TensorRT-LLM）深度集成，形成一键式模型压缩与部署流水线。

模型架构与推理协同设计。当前推理优化技术大多面向已有模型架构，未来可探索“推理友好型”模型设计，在预训练阶段引入量化感知、显存访问优化等约束，使模型天生更适配高效推理。例如，设计更规整的注意力头维度、减少算子种类、引入可预测的 KV Cache 访问模式等，从源头降低推理复杂度。

长上下文推理优化。随着大语言模型上下文窗口的不断扩展（如百万级 tokens），KV Cache 的存储与访存开销成为核心瓶颈。未来可探索更激进的 KV Cache 压缩技术，如注意力稀疏化、KV 合并、低秩近似等，结合分页管理与异步调度，实现超长上下文的实时推理。

5.4 结语

本次实训使我们深刻认识到：大语言模型推理优化并非单一技术能够解决，而是需要在模型压缩、调度策略、显存管理、硬件适配等多个层面协同发力。从 ZLLM 框架的搭建到 vLLM 系统优化，从动态批处理探索到量化策略创新，我们完成了从理论到实践、从基础到前沿的完整技术链路的探索。最终在实时榜单上的优异表现，验证了所采用优化策略的有效性，也为后续研究提供了坚实的实验基础。

大语言模型推理优化技术正处在快速发展的黄金时期，随着硬件算力的持续提升和算法创新的不断涌现，我们有理由相信，更高效、更智能、更普惠的大模型服务将在不远的未来成为现实。本实训所积累的经验与认知，将为后续的深入研究奠定坚实基础。